2019 最新 Python——GUI_tkinter 学习资料整理

所有资料均来自网络

人生苦短，我用 Python

目录

(一)：接触 Tkinter 7

(二)：Tkinter 的两大常用组件 Label 和 Button 10

(三)：Tkinter 的三个组件 Checkbutton、Radiobutton 和 Labelframe.. 14

Checkbutton 14
Radiobutton 16
Labelframe 18

(四)：Tkinter 组件 Entry 19

Entry 组件：输入框 19

(五)：Tkinter 组件 Listbox、Scrollbar 和 Scale 29

Listbox 29
Scrollbar 滚动条组件 30
Scale 组件 32

(六)：Tkinter 的 Text 组件 34

目录： 34
简单用法 35
插入图片和组件 35
插入按钮 35
插入图片 37
Indexs 索引 38
line.column 38
line.end 39
INSERT 40
CURRENT 40
END(或"end") 40
user-defined marks 40

User-defined tags("tag.first", "tag.last")....40
selection(SEL_FIRST, SEL_LAST) 41
window coordinate("@x,y") 41
embedded object name(window, images)....41
expressions 41
Marks 用法 43
Tags 用法 46
设置样式 47
设置 Tag 优先级 50
事件绑定 51

(七)：Tkinter 的 Text 组件实用技巧 53

①判断内容是否发生变化 53
②查找操作 54
③Text 组件还支持恢复和撤销的操作 55
自定义撤销操作 57

(八)：Tkinter 的 Canvas 组件 1.... 58

绘制对象 58
修改对象 59
显示文本 60
画圆或椭圆 61
画多边形 62

(九)：Tkinter 的 Canvas 组件 2.... 64

Canvas 组件支持对象 65
坐标系 65
画布对象显示的顺序 65
指定画布对象 66

(十)：Tkinter 的组件 Menu、Menubutton 和 OptionMenu 67

创建一个顶级菜单 67
实现下拉菜单和级联菜单： 68

实现弹出菜单 69
一个带有 Checkbutton 和 Radiobutton 的菜单 70
Menubutton 组件 72
选择菜单 73
星号* 作用 74
一、星号（*）作为形参，表示调用可变参数函数： 75
二、星号（*）作为实参时，表示通过解包参数调用函数 75

(十一)：Tkinter 事件绑定 77

目录 77
实例 78
捕获鼠标点击的位置的演示 78
关于键盘的演示 79
利用<Motion>事件获取鼠标实时位置的方式 80
事件序列 81
type 81
modifier 83
Event 对象 83
Key names 83

(十二)：Tkinter 的组件 Message 87

目录 87
实例 87
何时使用 Message 组件？ 88
用法 88
参数 88

(十三)：Tkinter 的组件 Spinbox 90

用法 90
参数 91

(十四)：Tkinter 的组件 PanedWindow 95

目录 95

何时使用 PanedWindow 组件？ 95
用法 95
参数 98
方法 99
forget(child) 99
identify(x, y) 99
panecget(child, option) 100
panecget(child, option) 100
paneconfig(child, **option) 100

(十五)：Tkinter 的组件 Toplevel 及 Tk 和 Toplevel 的方法 101

目录 101
何时使用 Toplevel 组件？ 101
Tk（根窗口）和 Toplevel（顶级窗口）的方法 102
attribute(*args)之 args 参数 106

(十六)：Tkinter 的三大布局管理器 pack、grid 和 place 107

目录 107
何时使用 pack 管理器？ 108
何时使用 grid 管理器？ 110
如何使用 place 管理器： 112
pack 的方法 115
pack(**options) 115
pack_configure(**options) 115
pack_forget() 115
pack_info() 116
pack_propagate(flag) 116
pack_slaves() 116
grid 的方法： 116
grid(**options) 116
grid_bbox(column=None, row=None, col2=None, row2=None) 117
grid_columnconfigure(index, **options) 117
grid_configure(**options) 117

grid_forget()....117
grid_info()....117
grid_location(x, y)....118
grid_propagate(flag)....118
grid_remove()....118
grid_rowconfigure(index, **options)....118
grid_size()....118
grid_slaves(row=None, column=None)....118
place 的方法....118
place(**options)....119
place_configure(**options)....119
place_forget()....120
place_info()....120
place_slaves()....120
slaves()....120

（十七）：Tkinter 的标准对话框模块 messagebox 、filedialog 和 colorchooser....121

目录....121
Messagebox（消息对话框）....121
函数参数....123
函数返回值....124
filedialog（文件对话框）....124
函数参数....125
filetypes 图解：....126
函数返回值....126
colorchooser（颜色对话框）....126
函数参数....127
函数返回值....128
Q&A....128

(一)：接触 Tkinter

Tkinter 是 Python 的标准 GUI 库

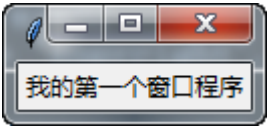
Tkinter（Tk + interface）实际上是建立在 Tk 技术上的。Tk 技术最初是为 TCL 设计的，TCL 是一门有名的工具命令语言。但是由于可移植性和灵活性非常高，加上非常容易使用，因此被移植到许多脚本语言里，包括 Perl、Ruby 和 Python。

Tkinter 是 Python 的默认 GUI 库，像 IDLE 就是使用 Tkinter 设计出来的，因此我们只需要在 IDLE 里导入 Tkinter 就可以使用了。

下面是一个简单 GUI 程序

```
1.  import tkinter as tk
2.
3.  #实例化一个 Tk，用于容纳整个工艺程序
4.  app = tk.Tk()  #Tk 类生成了一个顶层窗口的实例 app。是 top level 级别的一个窗口，也是 root 窗口
5.
6.  #设置它的标题栏
7.  app.title("My first python GUI code programmed with Tkinter")  #设置窗口的标题，IDLE 的标题就是文件的路径
8.
9.  #设置 Label 组件，Label 组件是最常用的组件之一。可以用于显示文本图标图片
10. theLabel = tk.Label(app, text='我的第一个窗口程序')  #Label()是一个组件，组件实例化以后成为一个组件对象，赋值到一个变量里面去
11. theLabel.pack()  #pack()方法用于自动调节组件自身的尺寸以及位置
12. #执行到这里，窗口还不会显示，必须执行下一句
13.
14.
15. app.mainloop()  #mainloop()是窗口的主事件循环，一般是放在整个工艺程序的最后一行，因为进入主事件循环后，就由 Tkinter 接管一切
16. #在界面编程里有这样一句名言：Don't call me, I'll call you。意思是一旦进入主事件循环，就不由我们写的代码做主了，而是由工艺程序来响应用
```

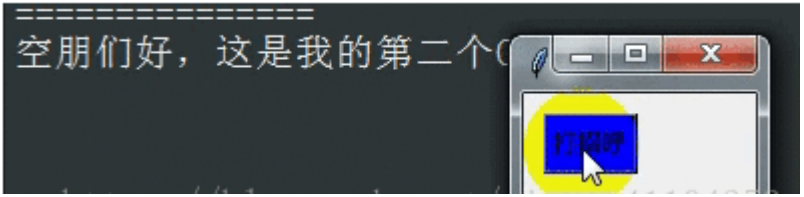
户的输入。比如说用户点击一个按钮，Tkinter 感受到这个按钮被点到了，就会调用我们为这个按钮安排好的方法，是由 Tkinter 调用我们的函数。



通常我们要写一个比较大的程序，应该把代码封装起来。
在面向对象的编程语言里就是封装成类。
下面是一个进阶版的，把 GUI 封装成类：

```
import tkinter as tk

class APP:
    def __init__(self, master):
        frame = tk.Frame(master)  #Frame 框架是放在顶层窗口里的。框架一般用于在复杂的布局里面将这些组件分组的（编者话：不知道是不是和 Java 里面的 Panel 起到同样的作用）
        frame.pack(side=tk.LEFT, padx=10, pady=10) #side 共有四个参数：Right，LEFT，TOP，BOTTOM
        #padx, pady 设置框架距离顶层窗口 x 轴，y 轴的距离

        #注意要加上 command 参数，参数值为函数名。当按钮被按下时，就会调用该方法
        self.hi_there = tk.Button(frame, text='打招呼', bg='black', fg='blue', command=self.say_hi)  #创建一个按钮组件，放在 Frame 框架里。bg 设置背景色的颜色。fg 是前景色的意思（英文没听清楚），把打招呼的颜色设置为蓝色.
        self.hi_there.pack()

    def say_hi(self):
        print('空朋们好，这是我的第二个 GUI 程序。')

```

```
root = tk.Tk()    #先创建一个顶层窗口
app = APP(root)   #再实例化这个 APP

root.mainloop()
```



(二)：Tkinter 的两大常用组件 Label 和 Button

Label 组件用于在界面上输出描述性的标签，这可以使用 Label 组件的文本输出功能来简单实现。

但是 Label 组件还可以显示图片，下面是一个关于这个的例子：

注意：PhotoImage() 方法传入的图片只支持 .gif 格式的

```
from tkinter import *   #把 Tkinter 模块里所有的东西都导进来了

root = Tk()

textLabel = Label(root, text='可爱小猫
', justify=LEFT, padx=10)  #在字符串里面加上\n 可以分两行显示
#justify 选项可以设置左对齐，右对齐。默认是 CENTER 居中
#padx 设置距离左边界的距离

textLabel.pack(side=LEFT)

photo = PhotoImage(file=r"C:\image\cat_200_200.gif")  #实例化 PhotoImage 就可以得到一个图片对象.传入想要的图片路径就可以了，只支持 GIF
imgLabel = Label(root, image=photo) #image 的值是 tkinter 的图片对象
imgLabel.pack(side=RIGHT)

mainloop()
```

有些时候，我们并不希望图片和文字分开。比如说想将图片作为背景，文字显示在图片的上面。这只用设置 Label 组件的 compound 属性就可以了。

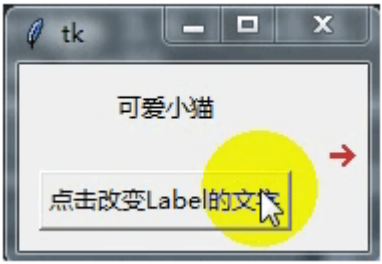
```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  photo = PhotoImage(file=r"C:\image\cat_200_200.gif")
6.  theLabel = Label(root,
7.                   text="好好学习\n 天天向上",
8.                   justify=LEFT,
9.                   image=photo,
10.                  compound=CENTER,   #混合属性
11.                  font=("宋体", 20),  #设置字体，直接写字体的名字就行，后面的 20 是字号，即字体大小
12.                  fg="white")
13.
14. theLabel.pack()
15.
16. mainloop()
```

Button 组件的绝大多数选项是和 Label 一样的，不过 Button 有一个功能就是接受用户的信息。Button 组件可以通过按钮上的文字和图标告诉用户按下去会发生什么，有什么功能。

Button 组件有一个叫做 command 的选项，用于指定一个函数或者方法。当用户点下按钮的时候，tkinter 就会自动的去调取这个函数或者方法。
修改第一个例子，添加一个按钮，在按钮被按下的时候将 Label 文本改变。
想要文本发生改变，只需要设置一个 textvariable 的选项就可以了。

```
1.  from tkinter import *
2.
3.
4.  def callback():
5.      var.set('可爱小狗')
6.
7.
8.  root = Tk()
9.
10. frame1 = Frame(root)
11. frame2 = Frame(root)
12.
13.
14. var = StringVar()     #实例化一个 StringVar()，得到一个 tkinter 的
    变量
15. var.set('可爱小猫')  #设置 var 的内容
16. textLabel = Label(frame1,   #放在框架一上
17.                   textvariable=var,  #改变原来的 text 选项，原来
    的 text 选项只能显示一个字符串，而 textvariable 可以显示一个变量，这
    个变量必须是 tkinter 的变量，tkinter 的字符串变量叫做 StringVar
18.                   justify=LEFT)
19. textLabel.pack(side=LEFT)
20.
21. photo = PhotoImage(file=r"C:\image\cat_200_200.gif")
22. imgLabel = Label(root, image=photo)
23. imgLabel.pack(side=RIGHT)
24.
25.
26. theButton = Button(frame2, text='点击改变 Label 的文字
    ', command=callback)
27. theButton.pack()
28.
29. frame1.pack(padx=10, pady=10)
```

```
30. frame2.pack(padx=10, pady=10)
31.
32. mainloop()
```

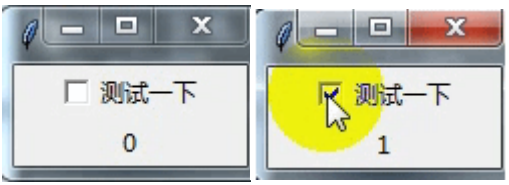


(三)：Tkinter 的三个组件 Checkbutton、Radiobutton 和 Labelframe

Checkbutton

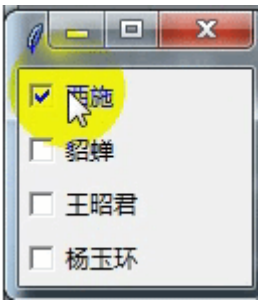
先感受一下 Checkbutton 的用法：

```
from tkinter import *

root = Tk()

v = IntVar()     #需要一个 tk 变量，用于表示按钮是否被选中。默认情况下，未选中是 0，选中是 1

c = Checkbutton(root, text='测试一下', variable=v)  #variable 用来表示按钮的状态（是否被按下）
c.pack()

l = Label(root, textvariable=v)  #为了更直观的看出选中和未选中 v 的表现状态，可以将其显示在 Label 标签里
l.pack()

mainloop()
```



下面是一个翻牌子的程序：

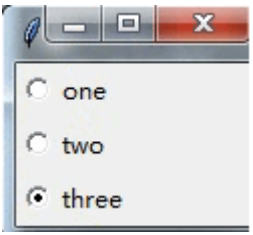
```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  GIRLS = ['西施', '貂蝉', '王昭君', '杨玉环']
6.
7.  v = []    #每一个按钮都需要一个存放变量的，所以这里用列表。后面用循环来添加
8.
9.  for girl in GIRLS:
10.     v.append(IntVar())#每一次都需要一个整形的 tk 变量用于每个存放按钮的状态，再追加到列表里去
11.     b = Checkbutton(root, text=girl, variable=v[-1]) #依次显示每个 girl 的名字。每一次都要拿到最后一个元素，因此是-1
12.     b.pack(anchor=W)  #所有的 Checkbutton 左对齐。anchor 用于指定显示位置，可以设置为 N W S E NW WS SE EN（东南西北英文缩写） CENTER 九个不同的值
13.
14. mainloop()
```



Radiobutton

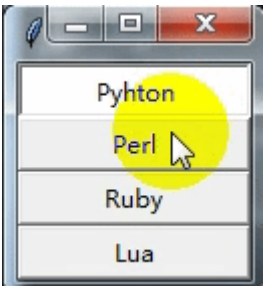
Radiobutton 的用法和 Checkbutton 的用法基本一致，唯一的不同是 Radiobutton 要实现单选的效果，即互斥的效果。同一组内，所有的 Radiobutton 只能共享一个 variable 这个选项，并且要设置不同的 value 的值。

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  v = IntVar()  #多个按钮只需要一个变量
6.
7.  #variable 只能是同一个变量。value 的值必须不同才能互斥
8.  Radiobutton(root, text='one', variable=v, value=1).pack(ancho
    r=W)
9.  Radiobutton(root, text='two', variable=v, value=2).pack(ancho
    r=W)
10. Radiobutton(root, text='three', variable=v, value=3).pack(anc
    hor=W)
11. #原理是每一次点中一个按钮，会把 value 的值给 v，给这个 v 说明点中了。
    再和其他的 value 值对比，都不是 2，因此不显示
12.
13. mainloop()
```



当存在多个选项的时候，使用循环会更加简洁。

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  v = IntVar()  #多个按钮只需要一个变量
6.
7.  LANGS = [
8.      ('Pyhton', 1),
9.      ('Perl', 2),
10.     ('Ruby', 3),
11.     ('Lua',  4)]
12.
13. v = IntVar()
14. v.set(1)
15.
16. for lang, num in LANGS:
17.     b = Radiobutton(root, text=lang, variable=v, value=num, i
    ndicatoron=False)  #改变前面的小圆点为按钮形式
18.     b.pack(fill=X)  #X 是横向填充，Y 是纵向填充
19.
20. mainloop()
```



Labelframe

Labelframe 标签框架。实际上是 Frame 框架的进化版，从形态上来说就是添加了 Label 的 Frame。有了它，Checkbutton 和 Radiobutton 的分组就变得简单了。

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  group = LabelFrame(root, text='最好的脚本语言是：
    ', padx=5, pady=5)
6.  group.pack(padx=10, pady=10)
7.
8.  LANGS = [
9.      ('Pyhton', 1),
10.     ('Perl', 2),
11.     ('Ruby', 3),
12.     ('Lua',  4)]
13.
14. v = IntVar()
15.
16. for lang, num in LANGS:
17.     b = Radiobutton(group, text=lang, variable=v, value=num)

18.     b.pack(anchor=W)
19.
20. mainloop()
```

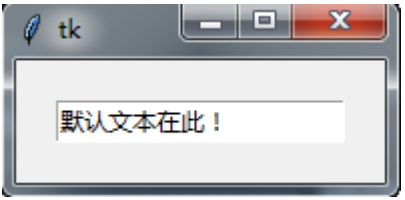
(四)：Tkinter 组件 Entry

Entry 组件：输入框

输入框是和程序打交道的一个途径。比如说程序要求输入账号和密码，就需要提供两个输入框，接受密码的输入框还会用*星号将实际内容隐藏起来。

学了好几个 Tkinter 的组件，不难发现，很多方法和选项之间是通用的，意思是名字一样，内容也一样。比如说，在输入框中用代码增加和删除内容，就是使用 insert() 和 delete() 方法。

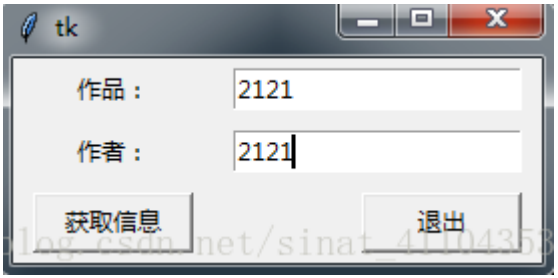
```
from tkinter import *

root = Tk()

e = Entry(root)
e.pack(padx=20, pady=20)

e.delete(0, END)  #将输入框里面的内容清空
e.insert(0, '默认文本在此！')

mainloop()
```



获取数据框里的内容，可以使用 Entry 组件里的 get 方法。也可以将 Tkinter 的变量，通常是字符串类型的变量挂钩到 textvariable 选项上，再通过变量的 get 方法获取也是可以的。

```
#tkinter 提供了三种布局组件的方式，第一种是 pack()，第二种是 Grid() 网格，第三种是 prase()
```

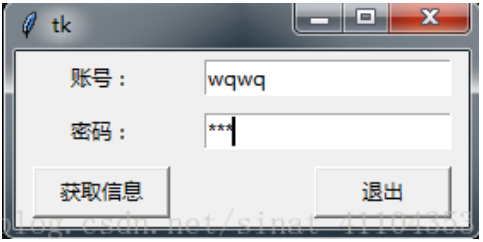
```
2.  #Grid 允许我们使用表格的形式管理组件
3.  from tkinter import *
4.
5.  root = Tk()
6.
7.  Label(root, text='作品：').grid(row=0, column=0)  #选项 row 代表
    行，column 代表列
8.  Label(root, text='作者：').grid(row=1, column=0)
9.
10.
11. e1 = Entry(root)
12. e2 = Entry(root)
13. e1.grid(row=0, column=1, padx=10, pady=5)
14. e2.grid(row=1, column=1, padx=10, pady=5)
15.
16. def show():
17.     print("作品：《%s》" % e1.get())
18.     print("作者：%s" % e2.get())
19.
20. Button(root, text='获取信息', width=10, command=show)\
21.             .grid(row=3, column=0, sticky=W, padx=10, pady=5
    ) #sticky 参数的值和 anceror 的一样，用来设置按钮的方位
22. Button(root, text='退出', width=10, command=root.quit)\
23.             .grid(row=3, column=1, sticky=E, padx=10, pady=5
    ) #退出直接调用根窗口的 quit 方法
24.
25. mainloop()
26.
27. #在 IDLE 里运行退出不了的原因是：IDLE 也是用 Tkinter 写出来的，这里会
    发生冲突。若是直接双击该文件打开就可以正常退出了。
```



实现密码框的用星号代替实际内容，可以通过 Entry 的 show 参数来实现：

```
1. from tkinter import *
2.
3. root = Tk()
4.
5. Label(root, text='账号：').grid(row=0, column=0)
6. Label(root, text='密码：').grid(row=1, column=0)
7.
8. v1 = StringVar()  #输入框里是字符串类型，因此用 Tkinter 的 StringVar 类型来存放
9. v2 = StringVar()  #需要两个变量来存放账号和密码
10.
11. e1 = Entry(root, textvariable=v1)
12. e2 = Entry(root, textvariable=v2, show='*') #想要显示什么就输入什么
13. e1.grid(row=0, column=1, padx=10, pady=5)
14. e2.grid(row=1, column=1, padx=10, pady=5)
15.
16. def show():
17.     print("账号：%s" % e1.get())
18.     print("密码：%s" % e2.get())
19.
20. Button(root, text='获取信息', width=10, command=show)\
21.             .grid(row=3, column=0, sticky=W, padx=10, pady=5)
```

```
Button(root, text='退出', width=10, command=root.quit)\
                .grid(row=3, column=1, sticky=E, padx=10, pady=5)

mainloop()
```



如果想要设计一个计算器，除了数字以外不允许其他字符出现，该怎么设计呢？

Entry 组件本身就自带了验证功能，就是可以验证输入框里面内容的合法性。实现该功能，就要设置选项 validate, validatecommand 和 invalidcommand

首先启用验证的" 开关" 是 validate 选项，该选项可以设置的值有：

```
focus      当 Entry 组件获得或失去焦点的时候验证（当 Entry 组件获得或失去焦点的时候调用 validate-command 指定的验证函数）
focusin    当 Entry 组件获得焦点的时候验证
focusout   当 Entry 组件失去焦点的时候验证
key        当输入框被编辑的时候验证
all        当出现上面任何一种情况的时候验证
none       ①关闭验证功能
           ②默认设置该选项（即不用启用验证）
           ③注意：是字符串的'none'，而非 None
```

其次是为 validatecommand 选项指定一个验证函数，该函数只能返回 True 或 False 表示验证的结果，一般情况下验证函数只需要知道输入框的内容即可，可以通过 Entry 组件的 get() 方法获得该字符串。

下边的例子中，在第一个输入框中输入 “我爱学习” 并通过 Tab 键将焦点转移到第二个输入框的时候，验证功能被成功触发。

```
1.  from tkinter import *
2.
3.  master = Tk()
4.
5.  def test():
6.      if e1.get() == '我爱学习':
7.          print('继续努力！')
8.          return True
9.      else:
10.         print('你在说什么鬼话')
11.         e1.delete(0, END)
12.         return False
13.
14.
15. v = StringVar()
16.
17. e1 = Entry(master, textvariable=v, validate='focusout', valid
    atecommand=test)  #当焦点移出的时候调用 validatecommand 设置的函
    数 test
18. e2 = Entry(master)
19. e1.pack(padx=10, pady=10)
20. e2.pack(padx=10, pady=10)
21.
22. mainloop()
```

invalidcommand 选项指定的函数只有在 validatecommand 的返回值为 False 的时候才会被调用。

下边的例子中，在第一个输入框输入不是 " 我爱学习 " 以外的字符串，并通过 Tab 键将焦点转移到第二个输入框，validatecommand 指定的验证函数被触发并返回 False，接着 invalidcommand 被触发：

```
1.  from tkinter import *
2.
3.  master = Tk()
4.
5.  def test1():
6.      if e1.get() == '我爱学习':
7.          print('继续努力！')
8.          return True
9.      else:
10.         print('你在说什么鬼话')
11.         e1.delete(0, END)
12.         return False
13.
14. def test2():
15.     print('invalidatecommand 设置的 test2 出场了')
16.
17.
18. v = StringVar()
19.
20. e1 = Entry(master, textvariable=v, validate='focusout', \
21.            validatecommand=test1, invalidcommand=test2)
22.
23. e2 = Entry(master)
24. e1.pack(padx=10, pady=10)
25. e2.pack(padx=10, pady=10)
26.
27. mainloop()
```

其实 Tkinter 还有隐藏功能，不过需要冷却才能实现：
Tkinter 为验证函数提供一些额外的选项：

```
1. %d   操作代码：0 表示删除操作；1 表示插入操作；2 表示获得、失去焦点或 textvariable 变量的值被修改
2.
3. %i   当用户尝试插入或删除操作的时候，该选项表示插入或删除的位置（索引号）如果是由于获得、失去焦点或 textvariable 变量的值被修改而调用验证函数，那么该值是-1
4.
5. %P   当输入框的值允许被改变的时候，该值有效，该值为输入框的最新文本内容
6.
7. %s   该值为调用验证函数前输入框的文本内容
8. %S   当插入或删除操作触发验证功能的时候，该值有效，该选项表示文本被插入和删除的内容
9.
10. %v   该组件当前的 validate 选项的值
11.
12. %V   调用验证函数的原因。该值是'focusin'、'focusout'、'key'或'focused'（textvariable 选项指定的变量值被修改）中的一个
13.
14. %W   该组件的名字（在 Tkinter 内部会注册一个名字，显示的是注册的名字，类似于句柄）
```

为了使用这些选项，我们可以这样写：validatecommand=(f, s1, s2, ...)---把它变成了一个元组

其中，f 就是我们 ‘冷却后’ 的验证函数名，s1, s2, s3 这些都是额外的选项，这些选项会作为参数依次传递给 f 函数。我们刚刚说了，使用额外功能前需要冷却，其实就是调用 register() 方法将验证函数包装起来：

```python
from tkinter import *

master = Tk()

def test(content, reason, name):  #test 不再是没有参数的方法了，参数是在 validatecommand 里出现的
    if content == '我爱学习':
        print('继续努力！')
        print(content, reason, name)
        return True
    else:
        print('你在说什么鬼话')
        print(content, reason, name)
        e1.delete(0, END)
        return False


v = StringVar()

testCMD = master.register(test)
e1 = Entry(master, textvariable=v, validate='focusout', \
           validatecommand=(testCMD, '%P', '%v', '%W'))
e2 = Entry(master)
e1.pack(padx=10, pady=10)
e2.pack(padx=10, pady=10)
mainloop()
```

下面是设计一个简单的加法计算器：

```
1. from tkinter import *
2. 
3. master = Tk()
4. 
5. frame = Frame(master)
6. frame.pack(padx=10, pady=10)
7. 
8. def test(content): #有些人可能会用输入框的 get 方法来获取内容，再通过将 validate 设置为 key 来判断输入参数合不合法。但是 validate 参数设置为 key 后，就不再能用输入框的 get 方法和 textvariable.get 方法获取输入内容。因为 validate 被指定为 key 时，有任何输入操作都会被拦截，然后调用验证函数，验证完后只有返回 True 才会将内容放到 textvariable 关联的变量中。
9.     return content.isdigit()    #字符串的内置方法，是数字返回 True；不是返回 False
10. 
11. 
12. v1 = StringVar()
13. v2 = StringVar()
14. v3 = StringVar()
15. 
16. testCMD = master.register(test)
17. e1 = Entry(frame, width=10, textvariable=v1, validate='key', \
18.            validatecommand=(testCMD, '%P')).grid(row=0, column=0)  #validate 的值是 key，那么输入框输入的内容如果是 True 会保留；是 False 会清除。设置输入框的宽度是 10 个字符
19. 
20. Label(frame, text='+').grid(row=0, column=1)
21. 
```

```
22. e2 = Entry(frame, width=10, textvariable=v2, validate='key', \
23.            validatecommand=(testCMD, '%P')).grid(row=0, column=2)
24.
25. Label(frame, text='=').grid(row=0, column=3)
26.
27. e3 = Entry(frame, width=10, textvariable=v3, state='readonly').grid(row=0, column=4)
28.
29. def calc():
30.     result = int(v1.get()) + int(v2.get())
31.     v3.set(str(result))
32.
33. Button(frame, text='计算结果', command=calc).grid(row=1, column=2, pady=5)
34.
35. mainloop()
```

(五)：Tkinter 组件 Listbox、Scrollbar 和 Scale

Listbox

Listbox 组件，用列表框的形式显示出选项，并且支持滚动条操作。所以更适用于提供大量选项。

```
from tkinter import *

master = Tk()

theLB = Listbox(master)
theLB.pack()

#theLB.insert(0, '') #向 Listbox 里添加选项，下标从零开始。最后可以用 END
for item in ['张三', '李四', '王五']:
    theLB.insert(END, item)   #END 表示最后一个。里面什么都没有时，END 是 0；插入一个后，END 是 1

'''''
theLB.delete(0, END)
#删除 Listbox 里所有的选项。删除操作有两个参数：起始位置和结束位置
#如果只有一个参数，就指定删除参数对应的选项
'''

#添加按钮组件选中哪个按下删除哪个
theButton = Button(master, text='删除它', \
                   command=lambda x=theLB:x.delete(ACTIVE)) #ACTIVE 表示当前选中的值
theButton.pack()

mainloop()
```

Listbox 组件根据 selectmode 选项提供了四种不同的选择模式：SINGLE（单选）、BROWSE（也是单选，但是拖动鼠标或通过方向键可以直接改变选项）、MUTIPLE（多选）和 EXTENDED（也是多选，但需要同时按住 Shift 键或 Ctrl 键或拖拽鼠标实现）。默认是 BROWSE。

Listbox 组件默认最多只能显示 10 个项目，如果有 11 个项目怎么解决？
可以通过在 Listbox 内滑动鼠标滚轮使最后一个现身，但这样很容易被忽略。

有两个方法可以避免：

①修改 height 选项。height 选项是用来设置 Listbox 显示的行数（不是像素），其默认值是 10
但是如果要显示项目过多，比如说 1000 个，修改 height 就不太合适了，因为总不能显示 1000 行
②可以为 Listbox 组件添加滚动条。

Scrollbar 滚动条组件

滚动条组件虽然是一个独立的组件，但几乎都是与其他组件一起配合使用。

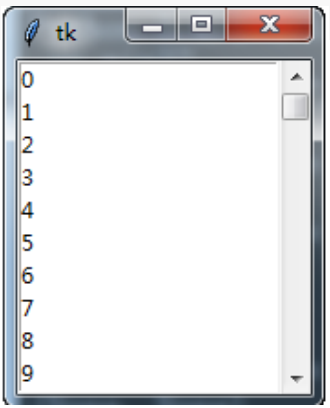
为了在某个组件上安装垂直滚动条，你需要做两件事：

1. 设置该组件的 yscrollcommand 选项为 Scrollbar 组件的 set() 方法

2. 设置 Scrollbar 组件的 command 选项为该组件的 yview() 方法

下面是一个在 Listbox 里添加滚动条的例子：

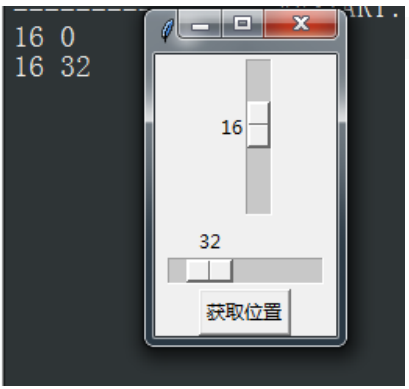
```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  sb = Scrollbar(root)
6.  sb.pack(side=RIGHT, fill=Y)  #把滚动条加到右边去，填充整个 y 轴
7.
8.  lb = Listbox(root, yscrollcommand=sb.set)
9.  #在 Listbox 里滑动鼠标滚轮时调用 set 方法，同时修改滚动条的位置
10.
11. for i in range(1000):
12.     lb.insert(END, i)
13. lb.pack(side=LEFT, fill=BOTH)
14.
15. #要与滚动条互通互连，要设置滚动条的 command 选项
16. sb.config(command=lb.yview)  #config 方法设置某个选项的值
17. #yview 方法是 Listbox 里的默认方法，方法已经设置好怎么显示垂直滚动对
    应的内容变化
18. #拖动垂直滚动条时调用 yview
19.
20. mainloop()
```



Scale 组件

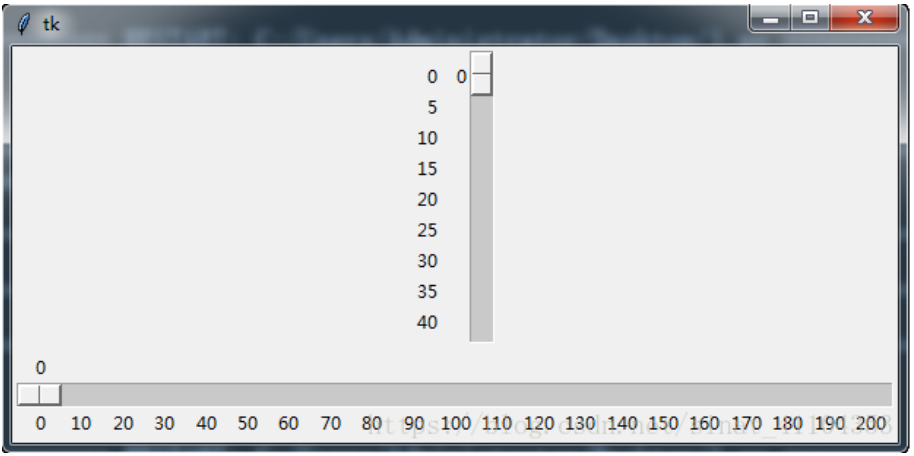
既然说到了滚动条，就继续来说说和滚动条组件很相似的一个组件 Scale 组件。Scale 组件主要通过滑块来表示某个范围内的一个数字。可以设置选项范围和分辨率（分辨率指的是步长，也就是精度）

```
1.  from tkinter import *
2.
3.  #当我们希望用户输入某个范围内的数字的时候，可能会使用 Entry 组件，但是 Entry 组件并不能限制一个范围
4.  root = Tk()
5.
6.  s1 = Scale(root, from_=0, to=42)  #有两个参数 from 和 to，但是 from 参数和 Python 的关键字冲突了，所以加个下横线
7.  s1.pack()
8.
9.  s2 = Scale(root, from_=0, to=200, orient=HORIZONTAL)  #默认 Scale 是垂直的，可以修改 orient 参数成水平的
10. s2.pack()
11.
12. #获取滑块当前位置使用的是 get 方法
13. def show():
14.     print(s1.get(), s2.get())
15.
16. Button(root, text='获取位置', command=show).pack()
17.
18. mainloop()
```



可以通过设置选项 tickinterval 和 resolution 改变步长和精度：

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  #tickinterval=5 每 5 个步长显示刻度
6.  #resolution 设置每次移动多少个长度，等于 5 的话，每次改变只能改变 5
7.  #设置 length 长度，不然都堆在一起了
8.  s1 = Scale(root, from_=0, to=42, tickinterval=5, resolution=5
    , length=200).pack()
9.  s2 = Scale(root, from_=0, to=200, tickinterval=10, orient=HOR
    IZONTAL, length=600).pack()
10.
11. mainloop()
```



(六)：Tkinter 的 Text 组件

绘制单行文本使用 Label 组件、多行选项使用 Listbox 组件、输入框使用 Entry 组件，显示和处理多行文本就需要使用我们的 Text 组件了。

Text 组件十分的强大和灵活，适用于多种任务。虽然说 Text 的主要目的是显示多行文本，但是也常常被当做简单的文本处理器、文本编辑器或者网页浏览器来使用。例如说我们的 IDLE 就是 Text 组件构成的。

目录：

简单用法

插入图片和组件

|--插入按钮

|--插入图片

Indexs 索引

|--line.column

|--line.end

|--INSERT

|--CURRENT

|--END(或"end")

|--user-defined marks

|--User-defined tags("tag.first", "tag.last")

|--selection(SEL_FIRST, SEL_LAST)

|--window coordinate("@x,y")

|--embedded object name(window, images)

|--expressions

Marks 用法

|--例一，Marks 事实上就是索引，用于表示位置：

|--例二，如果 Marks 前边的内容发生变化，那么 Mark 的位置也会跟着移动（说白了就是 Marks 会记住它后边的那货）：

|--例三，如果 Marks 周围的文本被删除了，Marks 任然还在（只是它后面的那货被删除了，所以它六神无主，只能初始化为"1.0"）：

|--例四，只有 mark_unset()方法可以解除对 Marks 的封印：

Tags 用法

|--设置样式

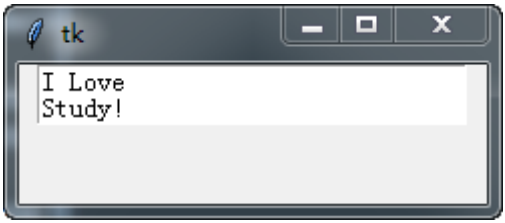
|--设置 Tag 优先级

|--事件绑定

简单用法

下面来演示一下 Text 组件的用法：

```
1. from tkinter import *
2. 
3. root = Tk()
4. 
5. text = Text(root, width=30, height=2) #30 的意思是 30 个平均字符的宽度，height 设置为两行
6. text.pack()
7. 
8. text.insert(INSERT, 'I Love\n')  #INSERT 表示输入光标所在的位置，初始化后的输入光标默认在左上角
9. text.insert(END, 'Study!')
10. 
11. mainloop()
12. #生成好的 Text 组件可以进行编辑（并不是只读形式的）
```

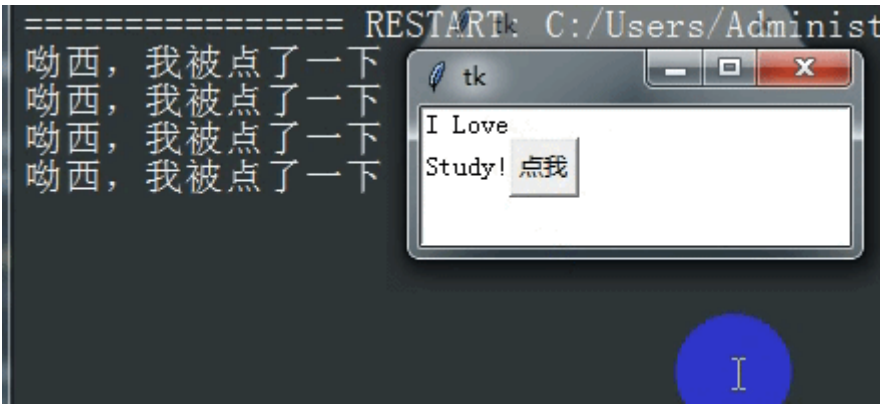


插入图片和组件

Text 组件不仅支持编辑和插入文本，还支持插入图片和组件。

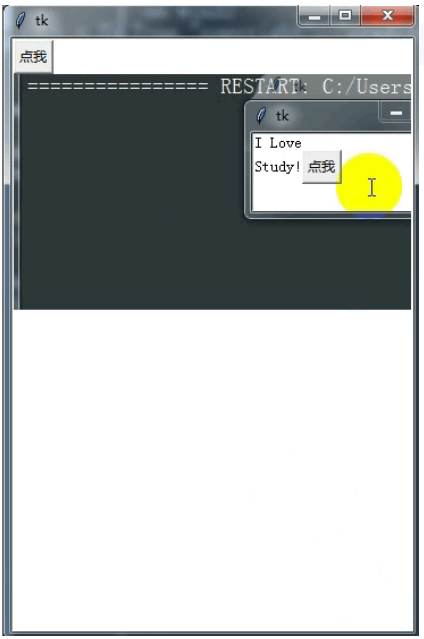
插入按钮

```python
from tkinter import *

root = Tk()

text = Text(root, width=30, height=5)
text.pack()

text.insert(INSERT, 'I Love\n')
text.insert(END, 'Study!')

def show():
    print('呦西，我被点了一下')

b1 = Button(text, text='点我', command=show) #注意放入的是 Text 而不是 root 了
text.window_create(INSERT, window=b1)

mainloop()
```



插入图片

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  text = Text(root, width=50, height=40)
6.  text.pack()
7.
8.  photo = PhotoImage(file="D:\LOTUS 专用图像\study.gif")  #只支持
    GIF 格式的
9.
10. def show():
11.     text.image_create(END, image=photo)
12.
13. b1 = Button(text, text='点我', command=show) #注意放入的是 Text
    而不是 root 了
14. text.window_create(INSERT, window=b1)
15.
16. mainloop()
```



Indexs 索引

至于想要在 Text 组件中定位，就要使用到 Indexs 索引了。

Indexs（索引是用来指向 Text 组件中文本的位置，跟 Python 的序列索引一样，Text 组件索引也是对应实际字符之间的位置。

Tkinter 提供一系列不同的索引类型：

索引类型	意思
line.column	行/列
line.end	某一行的末尾
INSERT	
CURRENT	
END	
user-defined marks	
user-defined tags("tag.first", "tag.last")	
selection(SEL_FIRST, SEL_LAST)	
window coordinate("@x,y")	
embedded object name(window, images)	
expressions	

line.column

由于 Text 支持多行文本，就是从一维空间变成了二维空间。因此可以用行和列定位一个位置。

那么，行列就是最基础的索引方式，他们将索引位置的行号和列号以字符串的形式表示出来（中间以"."分隔，例如"1.0"表示第一列第一行）。需要注意的是，行号以 1 开始，列号以 0 开始。你还可以使用以下语法构建索引：

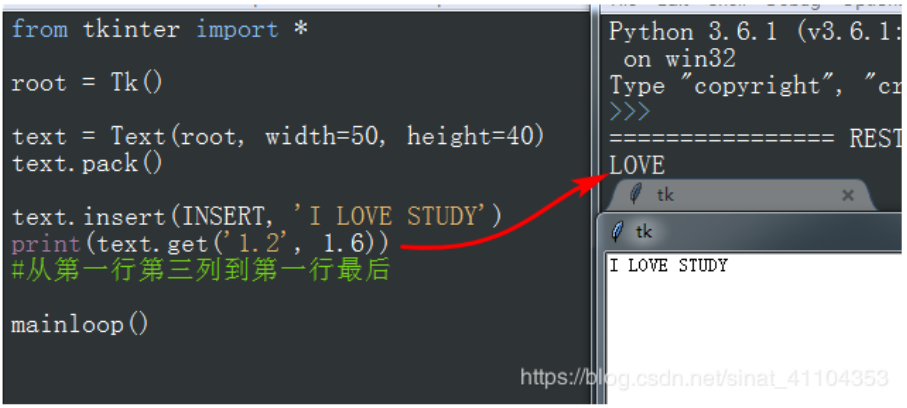
```
1.  "%d.%d" % (line, column)
```

指定超出现有文本的最后一行的行号，或超出一行中列数的列号都不会引发错误。对于这样的指定，Tkinter 解释为已有内容的末尾的下一个位置。

需要注意的是，使用行列的索引方式看起来像是浮点值。其实不只是像而已，你在需要指定索引的时候使用浮点值代替也是可以的：

```
1.  text.insert(INSERT, 'I LOVE STUDY')
2.  print(text.get('1.2', 1.6))  #1.2  1.6 可以不加双引号
```

使用 index() 方法可以将所有支持的“索引”格式（上面写的那一大堆，11 个）转化为“行/列”格式的索引号



line.end

行号加上字符串 ".end" 的格式表示为该行最后一个字符的位置

```
1.  text.insert(INSERT, 'I LOVE STUDY')
2.  print(text.get('1.2', '1.end'))  # end 为小写
3.  #从第一行第三列到第一行最后
```

```
from tkinter import *

root = Tk()

text = Text(root, width=50, height=40)
text.pack()

text.insert(INSERT, 'I LOVE STUDY')
print(text.get('1.2', '1.end'))
#从第一行第三列到第一行最后

mainloop()
```

```
Python 3.6.1
 on win32
Type "copyrigh
>>>
==============
LOVE STUDY
```

tk

tk

```
I LOVE STUDY
```

https://blog.csdn.net/sinat_41104353

INSERT

INSERT 和下面两个有点类似 C 的宏定义

INSERT (或 "insert")

对应插入光标的位置

CURRENT

CURRENT (或 "current") --用的少

对应与鼠标坐标最接近的位置。不过，如果你紧按鼠标任何一个按钮，它会直到你松开它才响应（即你松开时的位置）

END(或"end")

对应 Text 组件的文本缓冲区最后一个字符的下一个位置

user-defined marks

user-defined marks 是对 Text 组件中位置的命名。INSERT 和 CURRENT 是两个预先命名好的 marks，除此之外你可以自定义 marks

User-defined tags("tag.first", "tag.last")

User-defined tags 代表可以分配给 Text 组件的特殊事件绑定和风格。

你可以使用 "tag.first"（使用 tag 的文本的第一个字符之前）和 "tag.last"（使用 tag 的文本的最后一个字符之后）语法表示标签的范围。

```
1.  "%s.first" % tagname
2.  "%s.last" % tagname
```

如果查无此 tag，那么 Tkinter 会抛出 TclError 异常

selection(SEL_FIRST, SEL_LAST)

selection 是一个名为 SEL（或 "sel" ）的特殊 tag，表示当前被选中的范围，你可以使用 SEL_FIRST 到 SEL_LAST 来表示这个范围，如果没有选中的内容，那么 Tkinter 会抛出 TclError 异常

window coordinate("@x,y")

你还可以使用窗口坐标作为索引。例如在一个事件绑定中，你可以使用以下代码找到最接近鼠标位置的字符：

```
1.  "@%d,%d" % (event.x, event.y)
```

embedded object name(window, images)

embedded object name 用于指向在 Text 组件中嵌入的 window 和 image 对象。要引用一个 window，只要简单的将一个 Tkinter 组件实例作为索引即可。引用一个嵌入的 image，只需使用相应的 PhotoImage 和 BitmapImage 对象

expressions

expressions 用于修改任何格式的索引，用字符号的形式实现修改索引的表达式。
具体表达式实现如下：

表达式	含义
+ count chars	①将索引向前(->)移动 count 个字符 ②可以越过换行符，但不能超过 END 的位置
- count chars	①将索引向后(<-)移动 count 个字符 ②可以越过换行符，但不能超过"1.0"的位置
+ count lines	①将索引向前(->)移动 count 行 ②索引会尽量保持与移动前在同一列上，但如果移动后的那一行字符太少，将移动到该行的末尾

- count lines	①将索引向后(<-)移动 count 行 ②索引会尽量保持与移动前在同一列上，但如果移动后的那一行字符太少，将移动到该行的末尾
linestart	①将索引移动到当前索引所在行的起始位置 ②注意，使用该表达式前边必须有一个空格隔开
lineend	①将索引移动到当前索引所在行的末尾 ②注意，使用该表达式前边必须有一个空格隔开
wordstart	①将索引移动到当前索引指向的单词的开头 ②单词的定义是一系列字母、数字、下划线或任何非空白字符组合 ③注意，使用该该表达式前边必须有一个空格隔开
wordend	①将索引移动到当前索引指向的单词的末尾 ②单词的定义是一系列字母、数字、下划线或任何非空白字符组合 ③注意，使用该该表达式前边必须有一个空格隔开

ps.只要结果不产生歧义，关键字可以被缩写，空格也可以省略。例如："+ 5 chars" 可以简写成 "+5c"

在实现中，为了确保表达式为普通字符串，你可以使用 str 或格式化操作来创建一个表达式字符串。下面例子演示了如何删除插入光标前边的一个字符：

```
1.  def backspace(event):
2.      event.widget.delete("%s-1c" % INSERT, INSERT)
```

如果看书时发现一句美句，拿出笔和本，在本上写下这句话的位置（这是 Marks），并在书上对美句画横线（这是 tag）。

Marks 用法

Marks（标记）通常是嵌入到 Text 组件文本中的不可见对象。事实上 Marks 是指定字符串间的位置，并跟随相应的字符一起移动。Marks 有 INSERT，CURRENT 和 user-defined marks (用户自定义 Marks)。其中，INSERT 和 CURRENT 是 Tkinter 预定义的特殊 Marks，他们不能被删除。

INSERT(或 "insert") 用于指定当前插入光标的位置，Tkinter 会在该位置绘制一个闪烁的光标（因此，并不是所有的 Marks 都不可见）

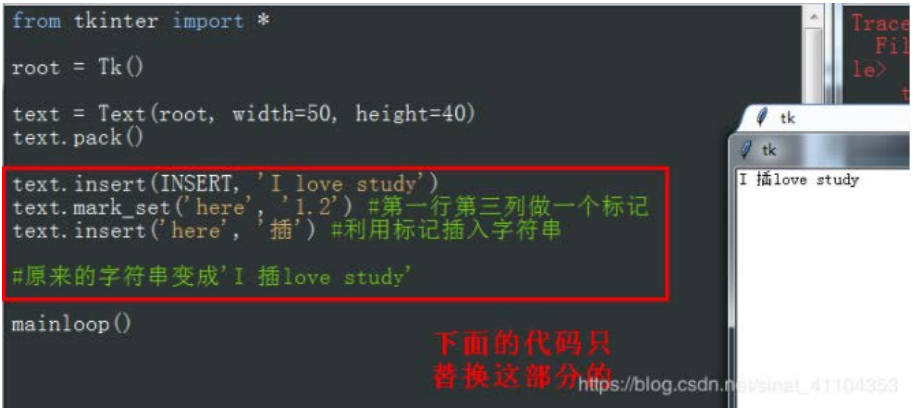
CURRENT(或 "current")用于指定对应与鼠标坐标最接近的位置。不过，如果你紧按鼠标任何一个按钮，它会直到你松开它才响应（即你松开时的位置）

你还可以自定义任意数量的 Marks，Marks 的名字是由普通字符串组成，可以是除了空白字符外的任何字符 (为了避免歧义，你应该起一个有意义的名字)。使用 mark_set() 方法创建和移动 Marks。

如果你在一个 Mark 标记的位置之前插入或删除文本，那么 Marks 会跟着一起移动。删除 Marks 你需要使用 mark.unset() 方法，删除 Marks 周围的文本并不会删除 Marks 本身

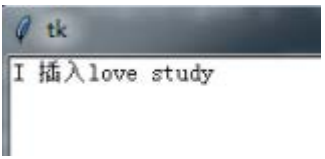
例一，Marks 事实上就是索引，用于表示位置：

```
1.  text.insert(INSERT, 'I love study')
2.  text.mark_set('here', '1.2') #第一行第三列做一个标记
3.  text.insert('here', '插') #利用标记插入字符串
4.
5.  #原来的字符串变成'I 插love study'
```



例二，如果 Marks 前边的内容发生变化，那么 Mark 的位置也会跟着移动（说白了就是 Marks 会记住它后边的那货）：

```
text.insert(INSERT, 'I love study')
text.mark_set('here', '1.2')
text.insert('here', '插')
text.insert('here', '入')

#原来的字符串变成'I 插入 love study'
```



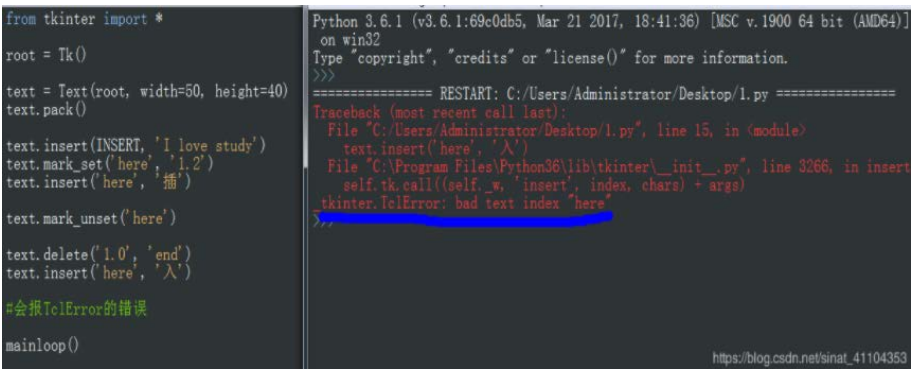
例三，如果 Marks 周围的文本被删除了，Marks 仍然还在（只是它后面的那货被删除了，所以它六神无主，只能初始化为"1.0"）：

```
text.insert(INSERT, 'I love study')
text.mark_set('here', '1.2')
text.insert('here', '插')

```

```
text.delete('1.0', 'end')
text.insert('here', '入')

#原来的字符串变成'入'
```

例四，只有 mark_unset() 方法可以解除对 Marks 的封印：

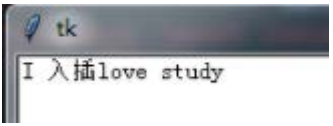
```
text.insert(INSERT, 'I love study')
text.mark_set('here', '1.2')
text.insert('here', '插')

text.mark_unset('here')

text.delete('1.0', 'end')
text.insert('here', '入')

#会报 TclError 的错误
```



默认插入内容到 Marks，是插入到它的左边（就是说插入一个字符的话，Marks 向后移动了一个字符的位置）。

那能不能插入到 Marks 的右侧呢（即 Marks 的位置一直不变）？其实是可以的，通过 mark_gravity() 方法就可以实现：

```
text.insert(INSERT, 'I love study')

text.mark_set('here', '1.2')
text.mark_gravity('here', LEFT)  #默认是 RIGHT，也就是说插入一个数据，Marks 默认是在这个数据的右边，即可能会（插入数据在 Marks 的右边时）相对插入前 Marks 的位置移动了一个。

text.insert('here', '插')
text.insert('here', '入')

#结果是"I 入插 love study"
```



不难总结出，Index 索引和 Marks 标记都是主要用于定位。Marks 可以看做是特殊的 Index，是一个可以自定义名字的 Index。但是他们又不是完全相同的，比如说在默认的情况下，在 Marks 指定的位置中插入数据，Marks 的位置会自动发生改变。

Tags 用法

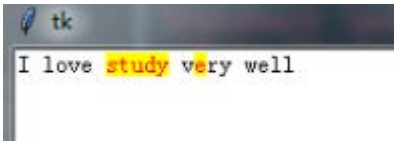
Tags（标签）通常用于改变 Text 组件中内容的样式和功能。你可以修改文本的字体、尺寸和颜色。另外，Tags 还允许你将文本、嵌入的组件和图片与鼠标和键盘等事件相关联。除了 user-defined tags (用户自定义的 Tags)，还有一个预定义的特殊 Tags：SEL。

SEL （或 "sel"）用于表示对应的选中内容（如果有的话）。

可以自定义任意数量的 Tags，Tags 的名字是由普通字符串组成，可以是除了空白字符以外的任何字符。另外，任何文本内容都支持多个 Tags 描述，任何 Tag 也可以用于描述多个不同的文本内容。

为指定文本添加 Tags 可以使用 tag_add() 方法：

```
1. text.insert(INSERT, 'I love study very well')
2. text.tag_add('tag1', '1.7', '1.12', '1.14')  #第一个参数是 tag 的名字，之后的参数是范围。这些参数表示的范围是'1.7'到'1.12'和一个单独的'1.14'
3. #光是这样只是在指定位置标注了一个 tag 标签，还需要对这些标签进行设置
4. text.tag_config('tag1', background='yellow', foreground='red')
5.
6. #字符串'I love study very well'中'study'和'e'颜色改变
```



设置样式

如上，使用 tag_config() 方法可以设置 Tags 的样式。下面罗列了 tag_config() 方法可以使用的选项：

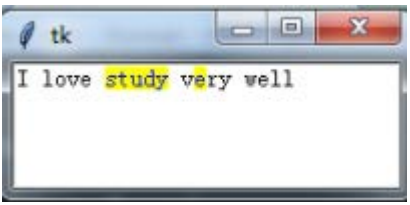
选项	含义
background	①指定该 Tag 所描述的内容的背景颜色 ②注意：bg 并不是该选项的缩写，在这里 bg 被解释成 bgstipple 选项的缩写
bgstipple	①指定一个位图作为背景，并使用 background 选项指定的颜色填充 ②只有设定了 background 选项，该选项才会生效 ③默认的标准位图有：'error', 'gray75', 'gray50', 'gray25', 'gray12', 'hourglass', 'info', 'questhead', 'question'和'warning'

borderwidth	①指定文本框的宽度 ②默认值是 0 ③只有设定了 relief 选项该选项才会生效 ④注意：该选项不能使用 bd 缩写
fgstipple	①指定一个位图作为前景色 ②默认的标准位图有：'error', 'gray75', 'gray50', 'gray25', 'gray12', 'hourglass', 'info', 'questhead', 'question'和'warning'
font	①指定该 Tag 所描述的内容使用的字体
foreground	①指定该 Tag 所描述的内容使用的前景色 ②注意：fg 并不是该选项的缩写，在这里 fg 被解释为 fgstipple 的缩写
justify	①控制文本的对齐方式 ②默认是 LEFT（左对齐），还可以选择 RIGHT（右对齐）和 CENTER（居中） ③注意：需要将 Tag 指向该行的第一个字符，该选项才能生效
lmargin1	①设置 Tag 指向的文本块第一行的缩进 ②默认值是 0 ③注意：需要将 Tag 指向该行的第一个字符或整个文本块，该选项才能生效
lmargin2	①设置 Tag 指向的文本块除了第一行其他行的缩进 ②默认值是 0 ③注意：需要将 Tag 指向整个文本块，该选项才能生效
offset	①设置 Tag 指向的文本相对于基线的偏移距离 ②可以控制文本相对于基线是升高（正数值）或者降低（负数值） ③默认值是 0
overstrike	①在 Tag 指定的文本范围画一条删除线 ②默认值是 False
relief	①指定 Tag 对应范围的文本的边框样式 ②可以使用的值有：SUNKEN，RAISED，GROOVE，RIDGE 或 FLAT ③默认值是 FLAT（没有边框）

margin	①设置 Tag 指向的文本块右侧的缩进 ②默认值是 0
spacing1	①设置 Tag 所描述的文本块中每一行与上方的文本间隔 ②注意：自动换行不算 ③默认值是 0
spacing2	①设置 Tag 所描述的文本块中自动换行的各行间的空白间隔 ②注意：换行符（"\n"）不算 ③默认值是 0
spacing3	①设置 Tag 所描述的文本块中每一行与下方的文本间隔 ②注意：自动换行不算 ③默认值是 0
tabs	①定制 Tag 所描述的文本块中 Tab 按键的功能 ②默认 Tab 被定义为 8 个字符的宽度 ③你还可以定制多个制表位：tabs=('3c', '5c', '12c')表示前三个 Tab 的宽度分别为 3cm，5cm，12cm，接着的 Tab 按照最后两个的差值计算，即：19cm，26cm，33cm ④你应该注意到，它上边'c'的含义是“厘米”而不是“字符”，还可以选择的单位有"i"(英寸)，"m"（毫米），"p"（DPI，大约是'1i'等于'72p'） ⑤如果是一个整型值，则单位是像素
underline	①该选项设置为 True 的话，则 Tag 所描述的范围内的文本将被画上下划线 ②默认值是 False
wrap	①设置当一行文本长度超过 width 选项设置的宽度时，是否自动换行。 ②该选项的值可以是：NONE（不自动换行），CHAR（按字符自动换行）和 WORD（按单词自动换行）

如果你对同一个范围内的文本加上多个 Tags（会形成一个栈的形式），并且设置相同的选项，那么新建的 Tag 样式会覆盖比较旧的 Tag：

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  text = Text(root, width=30, height=5)
6.  text.pack()
7.
8.  text.insert(INSERT, 'I love study very well')
9.  text.tag_add('tag1', '1.7', '1.12', '1.14')
10. text.tag_add('tag2', '1.7', '1.12', '1.14')
11. #注意这里是创建的顺序起决定性的作用
12.
13. text.tag_config('tag2', foreground='blue')
14. text.tag_config('tag1', background='yellow', foreground='red'
    )
15. #最后的前景色是蓝色，背景色还是黄色
16.
17. mainloop()
```



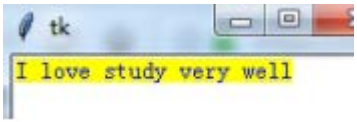
这里的前景蓝色有点不那么显眼，仔细看是蓝色的。

设置 Tag 优先级

你或许想控制 Tags 间的优先级，这可以实现么？

你可以使用 tag_raised() 和 tag_lower() 方法来提高和降低某个 Tag 的优先级。

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  text = Text(root, width=30, height=5)
6.  text.pack()
7.
8.  text.tag_config('tag2', foreground='blue')  #设置 tag 的选项时，
    如果没有该 tag，则会新建一个
9.  text.tag_config('tag1', background='yellow', foreground='red'
    ) #此时这个就是新建的 Tag 的顺序
10. #到这里是黄底红字的
11.
12. text.tag_lower('tag1')
13. #到这里就成了黄底蓝字
14.
15. text.insert(INSERT, 'I love study very well', ('tag2', 'tag1'
    ))  #和这里的 tag 选项的顺序无关。这样 Tag 加入的是整个文本
16.
17. mainloop()
```



事件绑定

另外 Tags 还支持事件绑定，使用的是 tag_bind() 的方法。

下边的例子中，我们将文本（"Baidu.com"）与鼠标事件进行绑定，当鼠标进入该文本段时，鼠标样式切换为 "arrow" 形态，离开文本段的时候切换回 "xterm" 形态。当触发鼠标“左键点击操作”时，使用默认浏览器打开百度首页

```python
from tkinter import *
import webbrowser

root = Tk()

text = Text(root, width=30, height=5)
text.pack()

text.insert(INSERT, 'Baidu.com 的创始人是李彦宏')

text.tag_add('link', '1.0', '1.8')
text.tag_config('link', foreground='blue', underline=True)

def show_arrow_cursor(event):
    text.config(cursor='arrow')

def show_xterm_cursor(event):
    text.config(cursor='xterm')

def click(event):
    webbrowser.open('http://www.baidu.com')

text.tag_bind('link', '<Enter>', show_arrow_cursor)  #<Enter>指的是当鼠标进入的时候调用 show_hand_cursor 函数
text.tag_bind('link', '<Leave>', show_xterm_cursor)
text.tag_bind('link', '<Button-1>', click)

mainloop()
```

(七)：Tkinter 的 Text 组件实用技巧

有关 Text 组件的基本用法在上一篇博客中已经讲述，这里不再赘述。
这里将讲述 Text 组件的实用技巧：

目录

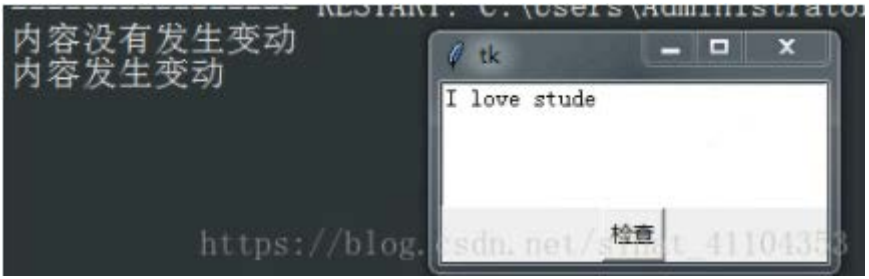
①判断内容是否发生变化
②查找操作
③Text 组件还支持恢复和撤销的操作
自定义撤销操作

①判断内容是否发生变化

比如说做一个笔记本（.txt 的那个记事本）程序，或者重新做一个 IDLE。它们都有输入框，有输入框当程序在关闭的时候应该自动检测内容是否发生改变，如果有改变但用户并没有保存的时候，应该提醒用户是否保存。
我们这个例子是通过验证文本的 md5 摘要来判断文本是否发生改变。

```
1.  from tkinter import *
2.  import hashlib  #获得 md5 的值要用该库
3.
4.  root = Tk()
5.
6.  text = Text(root, width=30, height=5)
7.  text.pack()
8.
9.  text.insert(INSERT, 'I love study')
10. contents = text.get('1.0', END)
11.
12. def getSig(contents):
13.     n = hashlib.md5(contents.encode())
14.     return n.digest()  #只需要得到 md5 值 n 的简单摘要即可
15.
16. sig = getSig(contents)
```

```
17.
18. def check():  #点下检查的时候，会重新获得一次 contents 的内容，将原来的 contents 进行一个 md5 的 hash，获得它的 md5 的值；同时也获得之后 contents 的 md5 的值，将两者进行对比。由于一个内容只能产生一个唯一的散列，因此若不一样，就说明内容发生了改变。
19.     contents = text.get('1.0', END)
20.     if sig != getSig(contents):
21.         print("内容发生变动")
22.     else:
23.         print("内容没有发生变动")
24.
25. Button(root, text="检查", command=check).pack()
26.
27. mainloop()
```

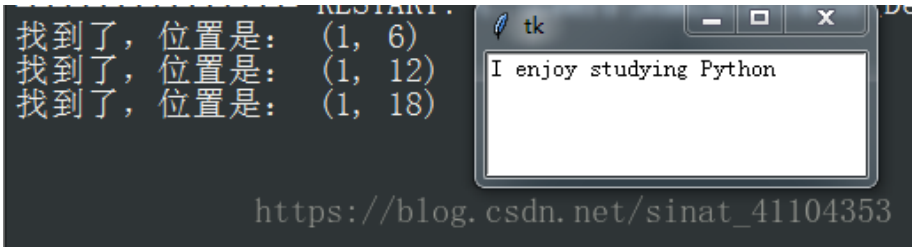


②查找操作

查找使用 search 方法，使用 search 方法可以对 text 的内容进行搜索。不过默认是只找到第一个位置。下面是一个可以全文搜索 字母 y 的例子：

```
1.  from tkinter import *
2.  import hashlib  #获得 md5 的值要用该库
3.
4.  root = Tk()
5.
6.  text = Text(root, width=30, height=5)
```

```
7.  text.pack()
8.
9.  text.insert(INSERT, 'I enjoy studying Python')
10.
11. def getIndex(text, index):   #要把任何支持的格式转化为默认行列字符串的格式可以使用 Text 组件的 index()方法
12.     return tuple(map(int, str.split(text.index(index), '.')))  #将转化好的转化成一个元组的形式返回
13.
14. start = '1.0'
15. while True:
16.     pos = text.search("y", start, stopindex=END)
17.     if not pos:  #找不到就跳出循环
18.         break
19.     print("找到了，位置是：", getIndex(text, pos)) #把找到的位置转化为人类能读懂的位置
20.     start = pos + '+1c'  #将 start 的位置指向下一个字符
21.
22. mainloop()
```



③Text 组件还支持恢复和撤销的操作

我们可以简单设置 Text 组件的 undo 选项为 True 开启其撤销功能，然后使用 edit_undo() 方法来实现撤销操作，用 edit_redo() 的方法来实现其恢复操作。

```python
from tkinter import *

root = Tk()

text = Text(root, width=30, height=5, undo=True)
text.pack()

text.insert(INSERT, 'I enjoy studying Python')

def show():
    text.edit_undo()

Button(root, text='撤销', command=show).pack()

mainloop()
```

这是因为 Text 组件内部有一个栈专门用于记录内容的每次变动，所以每次撤销操作就是一次弹栈操作，恢复就是再次压栈。

默认情况下，每一次完整操作将会放入栈中但怎么样才算是一次完整的操作呢？Tkinter 觉得每次焦点切换，用户按下 Enter 键。删除\插入操作的转换等之前的操作算是一次完整的操作。

自定义撤销操作

那我们能不能自定义呢？比如说我希望插入一个字符就算一次完整的操作，然后每次点击撤销就去掉一个字符。

当然可以，做法就是先将 autoseparators 选项设置为 False（因为这个选项是让 Tkinter 在认为一次完整的操作结束后自动插入 “分隔符”），然后绑定键盘事件，每次有输入就用 edit_separator() 方法人为地插入一个 “分隔符”：

```
1. from tkinter import *
2. 
3. root = Tk()
4. 
5. text = Text(root, width=30, height=5, undo=True, autoseparato
   rs=False)
6. text.pack()
7. 
8. text.insert(INSERT, 'I enjoy studying Python')
9. 
10. def callback(event):
11.     text.edit_separator()
12. 
13. text.bind('<Key>', callback)  #一旦有键盘事件就人为地插入一个分
   隔符
14. 
15. def show():
16.     text.edit_undo()
17. 
18. Button(root, text='撤销', command=show).pack()
19. 
20. mainloop()
```

(八)：Tkinter 的 Canvas 组件 1

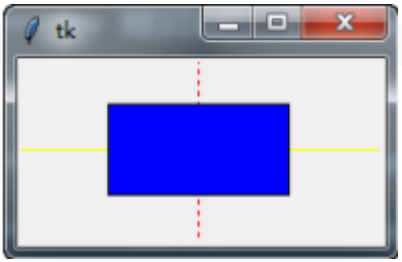
Canvas 组件是一个通用的组件，通常用于显示和编辑图形，我们可以用它来绘制直线三角形多边形甚至是绘制其他的组件，所以说 Canvas 组件可以让我们随心所欲的绘制界面。

绘制对象

在 Canvas 上绘制对象，我们可以使用 create 方法

```
1. from tkinter import *
2.
3. root = Tk()
4.
5. w =Canvas(root, width=200, height=100)  #width 和 height 单位是像素
6. w.pack()
7. #到这里还是看不见 Canvas 的，不过可以通过设置 Canvas 的 background 属性来改变背景颜色
8.
9. #create 后面加上下横线再加上我们要绘制对象的名字
10. w.create_line(0, 50, 200, 50, fill='yellow')  #前两个参数描述的是直线的起始位置是水平方向的第 0 个像素和竖直方向的第 50 像素。fill 设置填充颜色
11. w.create_line(100, 0, 100, 100, fill='red', dash=(4, 4))  #dash 是设置虚线
12. w.create_rectangle(50, 25, 150, 75, fill='blue')  #左上角右下角的坐标
13. #这三个画布对象没有意外的话会一直保留着，直到你去修改它们，它们就可能会被覆盖。
14. #例如说这条黄线本来是从左至右贯穿整个 Canvas 的，但是被矩形覆盖了，中间被覆盖的位置就看不到了
```

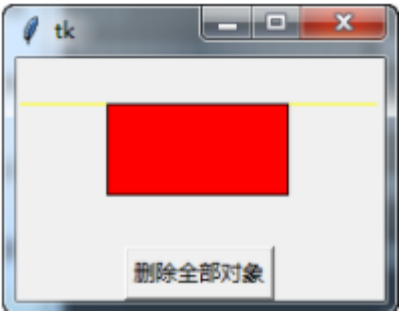
```
#还有一些方法可以对这些画布对象进行修改，如 Canvas 实例对象的 coords() itemconfig() move()方法,删除的话可以用 delete()方法。下例将进行演示

mainloop()
```



修改对象

下面将演示如何修改画布对象。使用的方法在上例的注释有提及

```
from tkinter import *

root = Tk()

w =Canvas(root, width=200, height=100)
w.pack()

line1 = w.create_line(0, 50, 200, 50, fill='yellow')
line2 = w.create_line(100, 0, 100, 100, fill='red', dash=(4, 4))
rect1 = w.create_rectangle(50, 25, 150, 75, fill='blue')

w.coords(line1, 0 ,25, 200, 25)  #用该方法把对象移动到一个新的位置
w.itemconfig(rect1, fill='red')  #主要是设置对象的选项
w.delete(line2)  #直接将对象的名字放进去就可以实现删除

Button(root, text='删除全部对象', command=(lambda x=ALL:w.delete(x))).pack()
#ALL 是一个 tag，表示所有的对象

mainloop()
```



显示文本

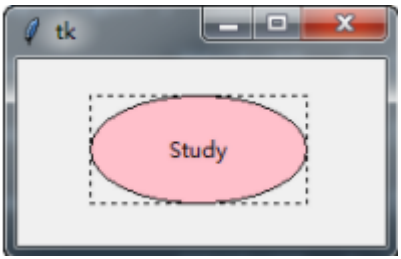
还可以在 Canvas 上显示文本：

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  w =Canvas(root, width=200, height=100)
6.  w.pack()
7.
8.  w.create_line(0, 0, 200, 100, fill='green', width=3) #线的宽度为 3 个像素
9.  w.create_line(200, 0, 0, 100, fill='green', width=3)
10. w.create_rectangle(40, 20, 160, 75, fill='green')
11. w.create_rectangle(65, 35, 135, 65, fill='yellow')
12.
13. w.create_text(100, 50, text='Study')  #可以修改 ancoror 选项使文本起始位置改变
14.
15. mainloop()
```

画圆或椭圆

在 Canvas 上画圆形或椭圆形

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  w =Canvas(root, width=200, height=100)
6.  w.pack()
7.
8.  w.create_rectangle(40, 20, 160, 80, dash=(4, 4))
9.  w.create_oval(40, 20, 160, 80, fill='pink')  #前面四个参数表示
    限定矩形所在的位置
10. #圆形就是限定矩形为正方形
11. #如 w.create_oval(70, 20, 130, 80, fill='pink')
12.
13. w.create_text(100, 50, text='Study')
14.
15. mainloop()
```



画多边形

Canvas 上画多边形可以使用 create_polygon 方法。下例是画一个五角星，首先要确定五个角的坐标：

```
1.  from tkinter import *
2.  import math as m
3.
4.  root = Tk()
5.
6.  w =Canvas(root, width=200, height=100)
7.  w.pack()
8.
9.  center_x = 100  #五角星中心点的坐标和半径
10. center_y = 50
11. r = 50
12.
13. points = [
14.     #左上点
15.     center_x - int(r * m.sin(2 * m.pi / 5)),
16.     center_y - int(r * m.cos(2 * m.pi / 5)),
17.     #右上点
18.     center_x + int(r * m.sin(2 * m.pi / 5)),
19.     center_y - int(r * m.cos(2 * m.pi / 5)),
20.     #左下角
21.     center_x - int(r * m.sin(m.pi / 5)),
22.     center_y + int(r * m.cos(m.pi / 5)),
23.     #顶点
24.     center_x,
25.     center_y - r,
26.     #右下点
27.     center_x + int(r * m.sin(m.pi / 5)),
28.     center_y + int(r * m.cos(m.pi / 5)),
29.     ]
```

```
30.
31. w.create_polygon(points, outline='green', fill='yellow') #方法把传入的两个点间连一条线，反复调用该方法看可以画出一个多边形
32.
33. mainloop()
```

(九)：Tkinter 的 Canvas 组件 2

画板实现原理：通过获取用户拖拽鼠标时候的每一个坐标，再将每一个坐标绘制成一个点就可以了。

遗憾的是，Tkinter 并没有为我们提供画点的方法，但是程序是死的，人是活的。我们可以通过绘制一个超小的椭圆来表示一个点。

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  w = Canvas(root, width=400, height=200, background='white')
6.  w.pack()
7.
8.  def paint(event):
9.      x1, y1 = (event.x - 1), (event.y - 1)
10.     x2, y2 = (event.x + 1), (event.y + 1)
11.     w.create_oval(x1, y1, x2, y2, fill='red')
12.
13. w.bind('<B1-Motion>', paint) #将画布与鼠标左键进行绑定
14.
15. Label(root, text='按住鼠标左键并移动，开始绘制
    ').pack(side=BOTTOM)
16.
17. mainloop()
```

Canvas 组件支持对象

arc（弧形、弦或扇形）
bitmap（内建的位图文件或 XBM 格式的文件）
image（BitmapImage 或 PhotoImage 的实例对象）
line（线）
oval（圆或椭圆形）
polygon（多边形）
rectangle（矩形）
text（文本）
window（组件）

其中，弦。扇形、椭圆形、圆形、多边形和矩形这些封闭式图形都是由轮廓线（设置 outline）和填充颜色（设置 fill）组成的，但都可以设置为透明（传入空字符串表示透明）

坐标系

由于画布可能比窗口大（带有滚动条的 Canvas 组件），因此 Canvas 组件可以选择使用两种坐标系：
窗口坐标系：以窗口的左上角作为坐标原点
画布坐标系：以画布的左上角作为坐标原点
将窗口坐标系转换为画布坐标系，可以使用 canvasx()或 canvasy()方法：

```
1. def callback(event):
2.     canvas = event.widget
3.     x = canvas.canvasx(event.x)
4.     y = canvas.canvasy(event.y)
5.     print(canvas.find_closest(x, y))
```

画布对象显示的顺序

Canvas 组件中创建的画布对象都会被列入显示列表中，越接近背景的画布对象位于显示列表的越下方。显示列表决定当两个画布对象重叠的时候是如何覆盖的（默认情况下新创建会覆盖就的画布对象的重叠部分，即位于显示列表上方的画布对象将覆盖下面那

个）。当然，显示列表中的画布对象可以被重新排序。（可以使用方法 lift() lower() tag_lower() tag_raise() 改变显示顺序）

指定画布对象

Canvas 组件提供几种方法让你指定画布对象：

Item handles

Tags

ALL

CURRENT

Item handles 事实上是一个用于指定某个画布对象的整型数字（也称为画布对象的 ID）。当你在 Canvas 组件上创建一个画布对象的时候，Tkinter 将自动为其指定一个在该 Canvas 组件中独一无二的整型值。然后各种 Canvas 的方法可以通过这个值操纵该画布对象。

Tags 是附在画布对象上的标签，Tags 由普通的非空白字符串组成。一个画布对象可以与多个 Tags 相关联，一个 Tag 也可以用于描述多个画布对象。然而，与 Text 组件不同，没有指定画布对象的 Tags 不能进行事件绑定和配置样式。也就是说，Canvas 组件的 Tags 是仅为画布对象所拥有。

Canvas 组件预定义了两个 Tags：ALL 和 CURRENT

ALL（或 "all"）表示 Canvas 组件中的所有画布对象

CURRENT（或 "current"）表示鼠标指针下的画布对象（如果有的话）

(十)：Tkinter 的组件 Menu、Menubutton 和 OptionMenu

Tkinter 提供了一个组件 Menu 用来实现顶级菜单，下拉菜单和弹出菜单。

目录

创建一个顶级菜单

实现下拉菜单和级联菜单：

实现弹出菜单

一个带有 Checkbutton 和 Radiobutton 的菜单

Menubutton 组件

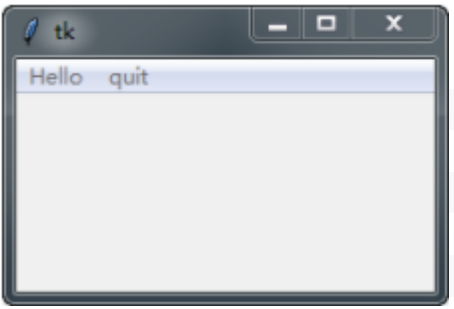
选择菜单

星号* 作用

一、星号（*）作为形参，表示调用可变参数函数：

二、星号（*）作为实参时，表示通过解包参数调用函数

创建一个顶级菜单




```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  def callback():
6.      print("你好")
7.
8.  menubar = Menu(root)
9.  menubar.add_command(label='Hello', command=callback)
10. menubar.add_command(label='quit', command=root.quit)
11.
12. root.config(menu=menubar)  #虽然 menu 已经创建，但是还没添加到 root 窗口中，所以要调用 config 方法对 root 的 menu 选项相关联
13. #在 IDLE 中点退出没反应是因为 IDLE 也是 Tkinter 实现的，他们两个共用了一个 mainloop()，退出代码重复了，产生了冲突
14.
15. mainloop()
```

实现下拉菜单和级联菜单：

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  def callback():
6.      print("你好")
7.
8.  menubar = Menu(root)
9.
10. filemenu = Menu(menubar, tearoff=False)
11. filemenu.add_command(label='打开', command=callback)
12. filemenu.add_command(label='保存', command=callback)
13. filemenu.add_separator()  #添加分割线
14. filemenu.add_command(label='退出', command=root.quit)
15. menubar.add_cascade(label='文件', menu=filemenu)   #创建级联菜单，menu 选项指定下一级的菜单是什么
16.
17.
18. editmenu = Menu(menubar, tearoff=False)
19. editmenu.add_command(label='剪切', command=callback)
20. editmenu.add_command(label='拷贝', command=callback)
21. editmenu.add_separator()  #添加分割线
22. editmenu.add_command(label='粘贴', command=callback)
23. menubar.add_cascade(label='编辑', menu=editmenu)
24.
25. root.config(menu=menubar)
26. #在 IDLE 中点退出没反应是因为 IDLE 也是 Tkinter 实现的，他们两个共用了一个 mainloop()，退出代码重复了，产生了冲突
27.
28. mainloop()
```

实现弹出菜单

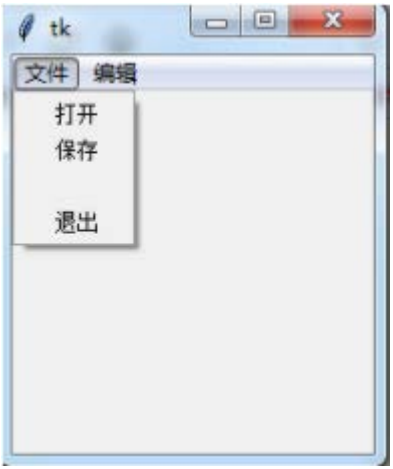
```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  def callback():
6.      print("你好")
7.
8.  menubar = Menu(root)
9.  menubar.add_command(label='撤销', command=callback)
10. menubar.add_command(label='重做', command=callback)
11.
12. frame = Frame(root, width=512, height=512)
13. frame.pack()
14.
15. def popup(event):
16.     menubar.post(event.x_root, event.y_root)
17.
18. frame.bind("<Button-3>", popup)
19.
20. mainloop()
```

点击右键出现如下菜单：

一个带有 Checkbutton 和 Radiobutton 的菜单

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  def callback():
6.      print("你好")
7.
8.  menubar = Menu(root)
9.
10. openVar = IntVar()
11. saveVar = IntVar()
12. quitVar = IntVar()
13.
14. filemenu = Menu(menubar, tearoff=False)
15. filemenu.add_checkbutton(label='打开
    ', command=callback, variable=openVar)
16. filemenu.add_checkbutton(label='保存
    ', command=callback, variable=saveVar)
17. filemenu.add_checkbutton()  #添加分割线
18. filemenu.add_checkbutton(label='退出
    ', command=root.quit, variable=quitVar)
19. menubar.add_cascade(label='文件', menu=filemenu)
20.
21.
22. editVar = IntVar()
23.
24. editmenu = Menu(menubar, tearoff=False)
25. editmenu.add_radiobutton(label='剪切
    ', command=callback, variable=editVar, value=1)
26. editmenu.add_radiobutton(label='拷贝
    ', command=callback, variable=editVar, value=2)
```

```
27. editmenu.add_separator()  #添加分割线
28. editmenu.add_radiobutton(label='粘贴
    ', command=callback, variable=editVar, value=3)
29. menubar.add_cascade(label='编辑', menu=editmenu)
30.
31. root.config(menu=menubar)
32. #在 IDLE 中点退出没反应是因为 IDLE 也是 Tkinter 实现的,他们两个共用了
    一个 mainloop()，退出代码重复了，产生了冲突
33.
34. mainloop()
```



Menubutton 组件

Menubutton 组件是一个和 Menu 组件相关联的按钮，事实上就是 Menu+Button。它可以在窗口的任何位置放一个 Button，点开后是一个 Menu。由于一开始并没有顶级菜单，所以用 Menubutton 实现一个个按钮在最上面，点开一个出来一个 Menu。不过现在 Tkinter 出现了顶级菜单，Menubutton 就没什么用处了。

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  def callback():
6.      print("你好")
7.
8.  mb = Menubutton(root, text='点我', relief=RAISED) #relief 设计按钮的样式
9.  mb.pack()
10.
11. filemenu = Menu(mb, tearoff=False)
12. filemenu.add_command(label='打开', command=callback)
13. filemenu.add_command(label='保存', command=callback)
14. filemenu.add_command()  #添加分割线
15. filemenu.add_command(label='退出', command=root.quit)
16.
17. mb.config(menu=filemenu)
18.
19. mainloop()
```

选择菜单

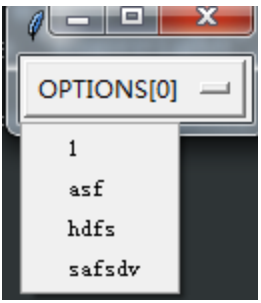
OptionMenu（选择菜单）组件。其实，Optionmenu 就是下拉菜单的一个改版，但是他的发明弥补了 Listbox（列表框）无法实现下拉列表框的遗憾，因为创建一个选择菜单的效果和创建下拉列表框的效果是一样的。

创建一个选择菜单非常简单，只需要给它一个 Tkinter 变量（用于记录用户选择了什么）以及若干选项即可：

```
1.  from tkinter import *
2.
3.  root = Tk()
4.
5.  variable = StringVar()
6.  variable.set('one')  #设置默认选中 one 选项
7.  #可以用该 Tkinter 变量的 get 方法获得用户点击的内容
8.
9.  w = OptionMenu(root, variable, 'one', 'two', 'three')  #关联一个 Tkinter 的变量 variable，表示用户选中了哪个值
10. #'one', 'two', 'three'表示的是选择菜单有哪些选项
11. w.pack()
12.
13. mainloop()
```

下面将演示如何将很多项添加到选择菜单中，就是当很多选项在元组或者列表中时，如何把他们添加到选择菜单中。

```
1.  from tkinter import *
2.
3.  OPTIONS = [
4.      '1',
5.      'asf',
6.      'hdfs',
7.      'safsdv'
8.      ]
9.
10. root = Tk()
11.
12. variable = StringVar()
13. variable.set('OPTIONS[0]')
14.
15. w = OptionMenu(root, variable, *OPTIONS)  #不加*整个列表会被作为一个选项
16. w.pack()
17.
18. mainloop()
```



星号* 作用

注意：星号（*）作为形参的时候是起到“打包”的作用，相反，作为实参的时候是起到“解包”的作用

一、星号（*）作为形参，表示调用可变参数函数：

通过在形参前加一个星号（*）或两个星号（**）来指定函数可以接受任意数量的参数：

```
>>> def fun1(*args):
    print(type(args))
    print(args)


>>> fun1(1, 2, 3, 4)
<class 'tuple'>
(1, 2, 3, 4)
```

```
>>> def fun2(**args):
    print(type(args))
    print(args)


>>> fun2(a=1, b=2, c=3, d=4)
<class 'dict'>
{'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

从两个示例可以看出：当参数形如 *args 时，传递给函数的任意个实参惠安为止打包成一个元组（tuple）；当参数形如 **args 时，传递给函数的任意个 key=value 实参会被包装进一个字典（dict）。

二、星号（*）作为实参时，表示通过解包参数调用函数

有打包就有解包，通过在实参前加一个星号（*）或两个星号（**）来对列表（list）、元组（tuple）或字典（dict）进行解包。

```
>>> def fun1(*args):
    print(type(args))
    print(args)


>>> a = [1, 2, 3, 4]
>>> b = (1, 2, 3, 4)

>>> fun1(*a)
<class 'tuple'>
(1, 2, 3, 4)

>>> fun1(*b)
<class 'tuple'>
(1, 2, 3, 4)
```

```
>>> def fun2(**args):
    print(type(args))
    print(args)


>>> c = {'one':1, 'two':2, 'three':3}
>>> fun2(**c)
<class 'dict'>
{'one': 1, 'two': 2, 'three': 3}
```

总结：一个星号（*）用来打包和解包序列，两个星号（**）用来打包和解包字典。

(十一)：Tkinter 事件绑定

一个 Tkinter 应用程序大部分时间是花在事件循环当中的，也就是通过 mainloop() 进入的事件循环。

事件可以有各种来源：用户触发的鼠标和键盘操作和窗口管理器触发的重绘事件（在多数情况下是由用户简介引起的）

Tkinter 给我们提供了一个强大的机制去管理这些事件，去定义这些事件的操作。对每一个组件来说，可以通过 bind() 的方法来将自己定义的函数或方法绑定到具体的事件上。

```
1. -widget.bind(event, handler)
```

当被触发的事件满足该组件绑定的事件的时候，Tkinter 就会带着事件对象（Event、事件本身的描述）去调用自定义的 handler() 方法。

目录

实例

捕获鼠标点击的位置的演示

关于键盘的演示

利用事件获取鼠标实时位置的方式

事件序列

type

modifier

Event 对象

Key names

实例

捕获鼠标点击的位置的演示

```
1. from tkinter import *
2.
3. root =Tk()
4.
5. #当触发<Button-1>，Tkinter 会带着事件本身去调用 callback，会把它传入到 callback 中，因此要个形参来接收对应的事件描述
6. def callback(event):
7.     print("点击位置：", event.x, event.y) #这个 x 和 y 表示的是相对于应用程序左上角的 x 和 y。root 的 x 和 y 相对的是屏幕
8.
9. frame = Frame(root, width=200, height=200)
10. frame.bind("<Button-1>", callback)  # -左边是事件本身，右边是事件详细描述。Button 表示的是鼠标的点击事件，1 表示的是左键，2 代表滚轮，3 表示右键，4、5 对于 Linux 系统才有用，表示滚轮向上滚和向下滚。对于 Windows 和 Mac 系统来说则是通过 mousewhell 来表示用户是向上滚还是向下滚了滚轮。
11. frame.pack()
12.
13. mainloop()
```

关于键盘的演示

```
1. from tkinter import *
2.
3. root =Tk()
4.
5. def callback(event):
6.     print(event.char)  #获取当前键盘按下的字符
7.
8. frame = Frame(root, width=200, height=200)
9. frame.bind("<Key>", callback)  #组件想要响应键盘事件，组件必须获得焦点，组件才会响应键盘来的消息。因为一个窗口可以有很多组件，键盘一次敲击不知道给哪个组件。
10. frame.focus_set()  #通过 focus_set 方法获得焦点。也可以设置 Frame 的 takefocus 选项为 True，然后使用 Tab 将焦点转移上来
11. frame.pack()
12.
13. mainloop()
```

利用<Motion>事件获取鼠标实时位置的方式

```
1. from tkinter import *
2.
3. root =Tk()
4.
5. def callback(event):
6.     print("当前位置: ", event.x, event.y)
7.
8. frame = Frame(root, width=200, height=200)
9. frame.bind("<Motion>", callback)  #Motion 事件表示当鼠标进入组件时，就会响应这个事件
10. frame.focus_set()
11. frame.pack()
12.
13. mainloop()
```

下方主要内容：

1.事件序列
2.type
3.modifier
4.Event 对象
5.Key names

事件序列

Tkinter 使用一种称为事件序列的机制来允许用户定义事件，用户需要使用 bind() 方法将具体的事件序列与自定义的方法相绑定。事件序列是以字符串的形式表示的，可以表示一个或多个相关联的事件（若果是多个事件，那么对应的方法只有在满足所有事件的前提下才会被调用）

事件序列使用以下语法描述：

```
1. <modifier-type-detail>
```

事件序列是包含在尖括号（<...>）中的
type 部分的内容是必须的，它通常用于描述普通的事件类型，例如鼠标点击或键盘按键点击 modifier 部分的内容是可选的，它通常用于描述组合键，例如 Ctrl + c（<Control-Key-C>），Shift + 鼠标左键点击（<Shift-Button-1>）
detail 部分的内容是可选的，它通常用于描述具体的按键，例如 Button-1 表示的是鼠标左键
如：
事件序列<Button-1>：用户点击鼠标左键
事件序列<KeyPress-H>：用户点击 H 按键
事件序列<Control-Shift-KeyPress-H>：用户同时点击 Ctrl + Shift +H

type

Active	当组件的状态从“未激活”变为“激活”的时候触发该事件
Button	当用户点击鼠标按键的时候触发该事件 detail 部分指定是具体用哪个键：<Button-1>鼠标左键，<Button-2>鼠标中键（滚轮点击），<Button-3>鼠标右键，<Button-4>滚轮上滚（Linux），<Button-5>滚轮下滚（Linux）

ButtonRelease	当用户释放鼠标按键的时候触发该事件 在大多数情况下，比 Button 要更好使用，因为如果当用户不小心按下鼠标键，用户可以将鼠标移出组件再释放鼠标，从而避免不小心触发事件
Configure	当组件的尺寸改变的时候触发该事件（窗口管理器触发的重绘事件，当你调整组件的尺寸或者移动应用程序，组件会和窗口一样被重绘）
Deactivate	当组件的状态从“激活”变为“未激活”的时候触发该事件 Destroy 当组件被销毁时触发该事件
Enter	当鼠标指针进入组件的时候触发该事件 注意：不是用户按下回车键（回车键是 Return<Key-Return>）
Expose	当窗口或组件的某部分不再被覆盖的时候触发该事件
FocusIn	当组件获得焦点的时候触发该事件 用户可以用 Tab 键将焦点转移到该组件上(需要该组件的 takefocus 选项为 True) 你也可以调用 focus_set()方法使该组件获得焦点
FocusOut	当组件失去焦点的时候触发该事件
KeyPress	当用户按下键盘按键的时候触发该事件 detail 可以指定具体的按键，例如<KeyPress-H>表示当大写字母 H 被按下的时候触发该事件 KeyPress 可以缩写为 Key
KeyRelease	当用户释放键盘按键的时候触发该事件
Leave	当鼠标指针离开组件的时候触发该事件
Map	当组件被映射的时候触发该事件 意思是在应用程序中显示该组件的时候，例如调用 get()方法
Motion	当鼠标在组件内移动的时候触发该事件
MouseWheel	当鼠标滚轮滚动的时候触发该事件 目前该事件仅支持 Windows 和 Mac 系统
Unmap	当组件被取消映射的时候触发该事件 意思是在应用程序中不再显示该组件的时候，例如调用 grid_remove()方法
Visibility	当应用程序至少有一部分在屏幕中是可见的时候触发该事件

modifier

在事件序列中，modifier 部分的内容可以是以下这些：

Alt	当按下 Alt 按键的时候
Any	表示任何类型的按键被按下的时候 例如<Any-keyPress>表示当用户按下任何按键时触发事件
Control	当按下 Ctrl 按键的时候
Double	当后续两个事件被连续触发的时候 例如<Double-Button-1>表示当用户双击鼠标左键时触发事件
Lock	当打开大写字母锁定键（CapsLock）的时候
Shift	当按下 Shift 按键的时候
Triple	跟 Double 类似，当后续三个事件被连续触发的时候

Event 对象

当 Tkinter 去回调你定义的函数的时候，就会带着 Event 对象（作为参数）去调用，Event 对象以下这些属性你可以使用：

widget	产生该事件的组件
x, y	当前的鼠标的位置坐标（相对于窗口左上角，单位为像素）
x_root, y_root	当前的鼠标的位置坐标（相对于屏幕左上角，单位为像素）
char	按键对应的字符（键盘事件专属，不会显示 Control，Shift 等键）
keysym	按键名，见下方 Key names（键盘事件专属）
keycode	按键码，见下方 Key names（键盘事件专属）
num	按钮数字（鼠标事件专属）
width, height	组件的新尺寸（Configure 事件专属）
type	该事件类型

Key names

当事件为<Key>，<KeyPress>，<KeyRelease>的时候，detail 可以通过设定具体的按键名(keysym)来筛选。例如<Key-H>表示按下键盘上的大写字母 H 时候触发事件，<Key-Tab>表示按下键盘上的 Tab 按键的时候触发事件。

下边列举了键盘所有特殊按键的keysym和keycode(按键码是对应美国标准101键盘的“Lantin-1”字符集，键盘标准不同，对应的按键码不同，但按键名是一样的)

按键名（keysym）	按键码（keycode）	代表的按键
Alt_L	64	左边的 Alt 按键
Alt_R	113	右边的 Alt 按键
BackSpace	22	BackSpace（退格）按键
Cancel	110	break 按键
Caps_Lock	66	CapsLock（大写字母锁定）按键
Control_L	37	左边的 Control
Control_R	109	右边的 Control
Delete	107	Delete 按键
Down	104	↓按键
End	103	End 按键
Escape	9	Esc 按键
Execute	111	SysReq 按键
F1	67	F1 按键
F2	68	F2 按键
F3	69	F3 按键
F4	70	F4 按键
F5	71	F5 按键
F6	72	F6 按键
F7	73	F7 按键
F8	74	F8 按键
F9	75	F9 按键
F10	76	F10 按键
F11	77	F11 按键
F12	96	F12 按键
Home	97	Home 按键
Insert	106	Insert 按键
Left	100	←按键
Linefeed	54	Linefeed（Ctrl + J）
KP_0	72	小键盘数字 0

31.	KP_1	73	小键盘数字 1
32.	KP_2	74	小键盘数字 2
33.	KP_3	75	小键盘数字 3
34.	KP_4	76	小键盘数字 4
35.	KP_5	77	小键盘数字 5
36.	KP_6	78	小键盘数字 6
37.	KP_7	79	小键盘数字 7
38.	KP_8	80	小键盘数字 8
39.	KP_9	81	小键盘数字 9
40.	KP_Add	86	小键盘的+按键
41.	KP_Begin	84	小键盘的中间按键（5）
42.	KP_Decimal	91	小键盘的点按键（.）
43.	KP_Delete	91	小键盘的删除键
44.	KP_Divide	112	小键盘的/按键
45.	KP_Down	88	小键盘的↓按键
46.	KP_End	87	小键盘的 End 按键
47.	KP_Enter	108	小键盘的 Enter 按键
48.	KP_Home	79	小键盘的 Home 按键
49.	KP_Insert	90	小键盘的 Insert 按键
50.	KP_Left	83	小键盘的←按键
51.	KP_Mutiply	63	小键盘的*按键
52.	KP_Next	89	小键盘的 PageDown 按键
53.	KP_Prior	81	小键盘的 PageUp 按键
54.	KP_Right	85	小键盘的→按键
55.	KP_Subtract	82	小键盘的-按键
56.	KP_Up	80	小键盘的↑按键
57.	Next	105	PageDown 按键
58.	Num_Lock	77	NumLock（数字锁定）按键
59.	Pause	110	Pause（暂停）按键
60.	Print	111	PrintScrn（打印屏幕）按键
61.	Prior	99	PageUp 按键
62.	Return	36	Enter（回车）按键
63.	Right	102	→按键

```
Scroll_Lock          78                ScrollLock 按键
Shift_L              50                左边的 Shift 按键
Shift_R              62                右边的 Shift 按键
Tab                  23                Tab（制表）按键
Up                   98                ↑按键
```

可以通过如下程序试验：

```python
from tkinter import *

root =Tk()

def callback(event):
    print(event.keysym)
    print(event.keycode)

frame = Frame(root, width=200, height=200)
frame.bind("<Key>", callback)  #组件想要响应键盘事件，组件必须获得焦点，组件才会响应键盘来的消息。因为一个窗口可以有很多组件，键盘一次敲击不知道给哪个组件。
frame.focus_set()  #通过 focus_set 方法获得焦点。也可以设置 Frame 的 takefocus 选项为 True，然后使用 Tab 将焦点转移上来
frame.pack()

mainloop()
```

(十二)：Tkinter 的组件 Message

Tkinter 的 Message（消息）组件：

Message 组件实际上是 Label 组件(只能显示单行)的变体,用于显示多行文本消息。Message 组件能够自动换行，并调整文本的尺寸使其适应给定的尺寸。

目录

实例
何时使用 Message 组件?
用法
参数

实例

下面的例子是演示自动换行的功能

```
1. from tkinter import *
2.
3. root = Tk()
4.
5. w1 = Message(root, text='这是一条短消息', width=100)
6. w1.pack()
7.
8. w2 = Message(root, text='这是一条超级超级超级超级无敌巨无
   霸长的消息', width=100)  #文本中也可以加上\n 进行强制换行
9. w2.pack()
10.
11. mainloop()
```

何时使用 Message 组件?

Message 组件用于显示简单的文本消息，通常你可以使用 Label 来代替。如果你希望使用多种字体来显示文本，那么应该使用 Text 组件。

用法

创建一个 Message 组件，所有你要做的事情就是指定要显示的文本内容。在必要的时候，该组件会自动换行，

参数

Message(master=None, **options)(class)

master----父组件

**options----组件选项，下方各表格详细列举了各个选项的具体含义和用法：

选项	含义
anchor	--控制文本消息的显示位置 --N, NE, E, SE, S, SW, W, NW 或 CENTER 来定位（EWSN 代表东西南北） --默认值是 CENTER
aspect	--设置高宽比，即宽度/高度的百分比的值 --默认值是 150（宽度比高度大 50%） --注意：如果设置了 width 选项的值，该选项将被忽略
background	--设置背景颜色 --默认值由系统指定
bg	--跟 background 一样
borderwidth	--指定边框高度 --默认值由系统指定，通常是 1 或 2 像素
bd	--跟 borderwidth 一样
cursor	--指定当鼠标在 Message 是飘过的时候的鼠标样式 --默认值由系统指定
font	--指定 Message 中文本的字体 --只能指定一种字体 --默认值由系统指定

foreground	--设置 Message 的文本的颜色 --默认值由系统指定
fg	--和 foreground 一样
highlightbackground	--指定当 Message 没有获得焦点的时候高亮边框的颜色 --默认值由系统指定，通常是标准背景颜色
highlightcolor	--指定当 Message 获得焦点的时候高亮边框的颜色 --默认值由系统指定
highlightthickness	--指定高亮边框的宽度 --默认值是 0（不带高亮边框）
justify	--定义如何对齐多行文本 --使用 LEFT，RIGHT 或 CENTER --注意，文本的位置取决于 anchor 选项 --默认值是 CENTER
padx	--指定水平方向上的额外间距（内容和边框间） --单位是像素
pady	--指定垂直方向上的额外间距（内容和边框间） --单位是像素
relief	--指定边框样式 --默认值是 FLAT --另外你还可以设置 SUNKEN，RAISED，GROOVE 或 RIDGE
takefocus	--如果是 True，该组件接受输入焦点 --默认值是 False
text	--指定 Label 显示的文本 --为了达到指定的高宽比（aspect 选项指定），文本内容将自动进行换行
textvariable	--Message 显示 Tkinter 的变量（通常是一个 StringVar 变量）的内容 --如果变量被修改，Message 的文本会自动更新
width	--设置 Message 的宽度 --单位是文本单位 --如果忽略该选项，将根据 aspect 设置的高宽比来设置合适的宽度

(十三)：Tkinter 的组件 Spinbox

组件 Spinbox（Tk8.4 新增）

Spinbox 组件是 Entry 组件（能让用户随意的输入数据）的变体，用于从一些固定的值（可以是连续的，也可以是一些字符串的组合）中选取一个。

何时使用 Spinbox 组件?
Spinbox 组件通常用于在限定数字中选取的情况下替代普通的 Entry 组件。
注意：Spinbox 仅支持 Python3.2 和 Tk8.4 以上的版本

用法

Spinbox 组件跟 Entry 组件的用法非常相似，主要区别是使用 Spinbox 组件，你可以通过范围或者元组指定允许用户输入的内容。

```
1. from tkinter import *
2.
3. root = Tk()
4.
5. w = Spinbox(root, from_=0, to=10)  #from 与 Python 的关键字冲突，故加上个下划线。步进可以设置
6. w.pack()
7.
8. mainloop()
```

你还可以通过元组指定允许输入的值

```
1. from tkinter import *
2.
3. root = Tk()
```

```
4.
5. w = Spinbox(root, values = (1, '2', '3', 'hi'))  #from
   与 Python 的关键字冲突，故加上个下划线
6. w.pack()
7.
8. mainloop()
```

参数

Spinbox(master=None, **options)(class)

master--父组件

**options--组件选项，下面是各个选项的具体含义和用法：

选项	含义
activebackground	设置当 Spinbox 处于 ACTIVE 状态下的背景颜色
background	设置背景颜色 默认值由系统指定
bg	跟 background 一样
borderwidth	设置边框宽度 默认值是 1 或 2 像素
buttonbackground	设置调节箭头的背景颜色
buttoncursor	指定当鼠标在调节箭头上方的鼠标样式
buttondownrelief	指定向下调节箭头的样式 默认值是 RAISED 还可以设置为 FLAT，SUNKEN，GROOVE 和 RIDGE
buttonup	指定向上调节箭头的样式 默认值是 RAISED 还可以设置为 FLAT，SUNKEN，GROOVE 和 RIDGE
command	指定一个函数，当用户点击调节箭头的时候将自动调用该函数 注意：当用户直接在输入框中输入数据时并不会触发该函数
cursor	指定当鼠标在 Spinbox 上飘过的时候的鼠标样式 默认值由系统指定
disabledbackground	设置当 Spinbox 处于 DISABLED 状态下的背景颜色

disabledforeground	设置当 Spinbox 处于 DISABLED 状态下的前景颜色
exportselection	指定选中的文本是否可以被复制到剪贴板 默认值是 True 可以修改为 False 表示不允许复制文本
font	指定 Spinbox 中文本的字体 默认值由系统指定
foreground	设置前景（文本）颜色 默认值由系统指定
fg	跟 foreground 一样
format	使用该选项设置选择数值的样式（from_和 to 指定范围，用户自行输入的不算） 例如使用 format='%10.4f'表示显示的数值占 10 位，小数点后保留 4 位
from_	该选项和 to 选项共同指定一个范围内的数值 increment 选项设置每次点击调节箭头递增递减的精度
highlightbackground	指定当 Spinbox 没有获得焦点的时候高亮边框的颜色 默认值由系统指定，通常是标准背景颜色
highlightcolor	指定当 Spinbox 获得焦点的时候高亮边框的颜色 默认值由系统指定
highlightthickness	指定高亮边框的宽度
increment	该选项指定当用户每次点击调节箭头的时候递增递减的精度 例如 from_=1, to=10, increment=0.5，那么每次用户点击调节箭头的时候，输入框中的数字递增递减 0.5
insertbackground	指定输入光标的颜色
insertborderwidth	指定输入光标的边框高度 如果被设置为非 0 值，光标样式会被设置为 RAISED 提示：将 insertborderwidth 设置的大一点才能看到效果
insertofftime	该选项控制光标的闪烁频率（灭） 单位是毫秒
insertontime	该选项控制光标的闪烁频率（亮） 单位是毫秒

insertwidth	指定光标的宽度 默认值是 1 或 2 像素
invalidcommand	指定当输入框的内容“非法”时调用的函数 也就是指定当validateCommand选项指定的函数返回False时的函数 详见 Entry 组件
invcmd	跟 invalidcommand 一样
justify	定义如何对齐多行文本 使用 LEFT，RIGHT 或 CENTER 默认值是 CENTER
readonlybackground	设置当 Spinbox 处于“readonly”状态下的背景颜色
relief	指定边框样式 默认值是 FLAT 另外你还可以设置 SUNKEN，RAISED，GROOVE 或 RIDGE
repeatdelay	该选项指定鼠标左键点击滚动条凹槽的响应时间 默认值是 400（毫秒）
repeatinterval	该选项指定鼠标左键紧按滚动条凹槽的响应间隔 默认值是 100（毫秒）
selectbackground	指定输入框的文本被选中时的背景颜色 默认值由系统指定
state	Spinbox 组件可以设置的状态：NORMAL，DISABLED 或“readonly”（注意，这个是字符串。它跟 DISABLED 相似，但他支持选中和拷贝，只是不能修改，而 DISABLED 是完全禁止） 默认值是 NORMAL 注意，如果此选项设置为 DISABLED 或”readonly”，那么调用 insert()和 delete()方法都会被忽略
takefocus	指定使用 Tab 键可以将焦点移动到输入框中 默认是开启的，可以将该选项设置为 False 避免焦点在此输入框中

textvariable	指定一个与输入框内容相关联的 Tkinter 变量（通常是 StringVar） 当输入框的内容发生改变时，该变量的值也会相应发生改变
to	该选项和 from_选项共同指定一个范围的数值 increment 选项设置每次点击调节箭头递增递减的精度
validate	该选项设置是否启用内容验证
validatecommand	该选项指定一个验证函数，用于验证输入框内容是否合法 验证函数需要返回 True 或 False 表示验证结果 注意，该选项只有当 validate 的值非'none'时才有效
vcmd	和 validatecommand 一样
values	提供两个方法限定用户输入的内容，一种是通过 from_和 to 选项设置范围，另一种则是将可选值以元组的形式赋给 values 选项 例如 values=("ds", "sad", "aesf")则允许用户在这四个字符串中选择 width：设置输入框的宽度，以字符为单位 默认值是 20 对于变宽字体来说，组件的实际宽度等于字体的平均宽度乘以 width 选项的值
wrap	默认情况下（False），当输入框中的值是第一个（最后一个）的时候，再点击向上（向下）调节箭头，内容不会改变。 当该选项的值设置为 True，则当输入框中的值是第一个（最后一个）的时候，再点击向上（向下）调节箭头，内容将回到最后一个（第一个）。 提示：其实就是开启循环的意思
xscrollcommand	与 scrollbar（滚动条）组件相关联 如果你觉得用户输入的内容会超过该组件的输入框宽度，那么可以考虑设置该选项

(十四)：Tkinter 的组件 PanedWindow

PanedWindow (窗格) 组件 (Tk8.4 新增是一个空间管理组件。跟 Frame 组件类似，都是为组件提供一个框架，不过 PanedWindow 允许让用户调整应用程序的空间划分。

目录

何时使用 PanedWindow 组件?

用法

参数

PanedWindow(master=None, **options)(class)

方法

add(child, **options)

forget(child)

identify(x, y)

panecget(child, option)

panecget(child, option)

paneconfig(child, **option)

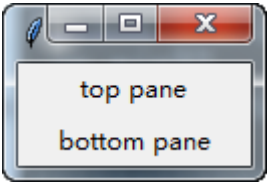
何时使用 PanedWindow 组件?

当你需要提供一个可供用户调整的多空间框架的时候，可以使用 PanedWindow 组件。PanedWindow 组件会为每一个子组件生成一个独立的窗格，用户可以自由调整窗格的大小。

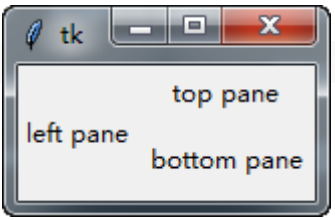
用法

```
from tkinter import *

m = PanedWindow(orient=VERTICAL)
m.pack(fill=BOTH, expand=1)

top = Label(m, text='top pane')
m.add(top)

```

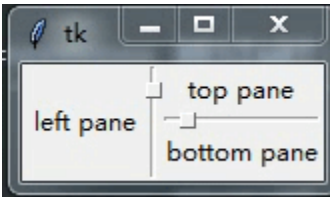
```
9.  bottom = Label(m, text='bottom pane')
10. m.add(bottom)
11. #拖拽鼠标的左键可以改变布局
12. #中间有条隐藏的线，是二窗格
13.
14. mainloop()
```




```
1.  #创建一个三窗格
2.  from tkinter import *
3.
4.  m1 = PanedWindow()  #默认是左右分布的
5.  m1.pack(fill=BOTH, expand=1)
6.
7.  left = Label(m1, text='left pane')
8.  m1.add(left)
9.
10. m2 = PanedWindow(orient=VERTICAL)
11. m1.add(m2)
12.
13. top = Label(m2, text='top pane')
14. m2.add(top)
15.
16. bottom = Label(m2, text='bottom pane')
17. m2.add(bottom)
18.
19. mainloop()
```



```
1. #显示手柄和分割线
2. from tkinter import *
3.
4. m1 = PanedWindow(showhandle=True, sashrelief=SUNKEN)  #
   默认是左右分布的
5. m1.pack(fill=BOTH, expand=1)
6.
7. left = Label(m1, text='left pane')
8. m1.add(left)
9.
10. m2 = PanedWindow(orient=VERTICAL, showhandle=True, sash
    relief=SUNKEN)
11. m1.add(m2)
12.
13. top = Label(m2, text='top pane')
14. m2.add(top)
15.
16. bottom = Label(m2, text='bottom pane')
17. m2.add(bottom)
18.
19. mainloop()
20. #分割线上的类似正方形的东西就是 handle
```



参数

PanedWindow(master=None, **options)(class)

master--父组件

**options--组件选项，下方详细列举了各个选项的具体含义和用法

选项	含义
background	设置背景颜色
bg	跟 background 一样
borderwidth	设置边框宽度
bd	跟 borderwidth 一样
cursor	指定当鼠标在 PanedWindow 上飘过的时候的鼠标样式 默认值由系统指定
handlepad	调节“手柄”的位置 例如 orient=VERTICAL，则 handlepad 选项表示“分割线”上的手柄与左端的距离 默认值是 8 像素
handlesize	设置“手柄”的尺寸（由于“手柄”必须是一个正方形，所以是设置正方形的边长），默认值是 8 像素
height	设置 PanedWindow 的高度 如果忽略该选项，则高度由子组件的尺寸决定
opaqueresize	该选项定义了用户调整窗格尺寸的操作 如果该选项的值为 True（默认），窗格尺寸随用户鼠标的拖拽而改变 如果该选项的值为 False，窗格的尺寸在用户释放鼠标的时候才更新到新的位置
orient	指定窗格的分布方式 可以是 HORIZONTAL（横向分布）和 VERTICAL（纵向分布）
relief	指定边框样式 默认值是 FLAT 另外你还可以设置 SUNKEN，RAISED，GROOVED 或 RIDGE
sashpad	设置每一条分割线到窗格间的间距

sashrelief	设置分割线的样式 默认值是 FLAT 另外你还可以设置 SUNKEN，RAISED，GROOVED 或 RIDGE
sashwidth	设置分割线的宽度
showhandle	设置是否显示调节窗格的手柄 默认值为 False
width	设置 PanedWindow 的宽度 如果忽略该选项，则高度由子组件的尺寸决定

方法

add(child, **options)

--添加一个新的子组件到窗格中

--下方表格列举了各个 options 选项的具体含义

选项	含义
after	添加新的子组件到指定子组件后边
before	添加新的子组件到指定子组件前边
height	指定子组件的高度
minsize	该选项控制窗格不得低于的值 如果 orient=HORIZONTAL，表示窗格的宽度不得低于该选项的值 如果 orient=VERTICAL，表示窗格的高度不得低于该选项的值
padx	指定子组件的水平间距
pady	指定子组件的垂直间距
sticky	当窗格的尺寸大于子组件时，该选项指定子组件位于窗格的位置 可选的值有：E、S、W、N（东南西北）以及他们的组合值
width	指定子组件的宽度

forget(child)

--删除一个子组件

identify(x, y)

--给定一个坐标（x, y），返回该坐标所在的元素名称

--如果该坐标位于子组件上，返回空字符串

--如果该坐标位于分割线上，返回一个二元组（n, 'sash'），n 为 0 表示第一个分割线
--如果该坐标位于手柄上，返回一个二元组（n, 'handle'），n 为 0 表示第一个手柄

panecget(child, option)
--获得子组件指定选项的值

panecget(child, option)
--获得子组件指定选项的值

paneconfig(child, **option)
--设置子组件的各种选项
--下面列举了各个 options 选项具体含义
after：添加新的子组件到指定子组件后边
before：添加新的子组件到指定子组件前边
height：指定子组件的高度

(十五)：Tkinter 的组件 Toplevel 及 Tk 和 Toplevel 的方法

Toplevel（顶级窗口）组件类似于 Frame 组件，但 Toplevel 组件是一个独立的顶级窗口，这种窗口通常拥有标题栏、边框等部件，和 Tk()创建出来的根窗口是一样的，共享着一样的方法。

目录

何时使用 Toplevel 组件?

Tk（根窗口）和 Toplevel（顶级窗口）的方法

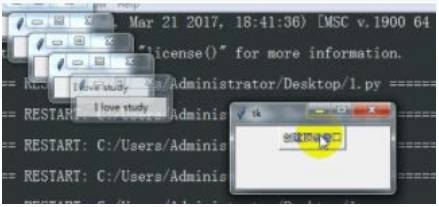
attribute(*args)之 args 参数

何时使用 Toplevel 组件?

Toplevel 组件通常用在显示额外的窗口、对话框或者其他弹出窗口上。

下面的例子，我们在 root 窗口添加一个按钮用于创建一个顶级窗口，点一下来一个。

```
1. from tkinter import *
2.
3. root = Tk()
4.
5. def create():
6.     top = Toplevel()
7.     top.title('Python')
8.
9.     msg = Message(top, text='I love study')
10.    msg.pack()
11.
12. Button(root, text='创建顶级窗口
    ', command=create).pack()
13.
14. mainloop()
```



Tk（根窗口）和 Toplevel（顶级窗口）的方法

下边这一系列方法用于与窗口管理器进行交互。他们可以被 Tk（根窗口）调用，同时也适用于 Toplevel（顶级窗口）。

注意：并非所有操作系统均完全支持下方所有方法的实现。

方法	含义
aspect(minNumber=None, minDenom=None, maxNumber=None, maxDenom=None,)	--控制该窗口的宽高比(width:height) --宽高比限制在：minNumber/minDenom~maxNumber/maxDenom --如果忽略参数，则返回一个 4 元组表示当前的限制（如果有的话）
attribute(*args) --例如设置窗口的透明度为 50%，可用 attribute("-alpha", 0.5)代替 attribute(alpha=0.5)	--设置和获取窗口属性 --如果你只给出选项名，将返回当前窗口该选项的值 --注意：以下选项不支持关键字参数，你需要在选项前加横岗（-）并用字符串的方式表示，用逗号（,）隔开选项和值。 --args 选项列表见文末
client(name=None)	--设置和获取 WM_CLIENT_MACHINE 属性 --如果要删除 WM_CLIENT_MACHINE 属性，赋值为空字符串即可 --该属性仅支持 X 窗口系统的窗口管理器，其他系统均忽略
colormapwindows(*wlist)	--设置和获取 WM_COLORMAP_WINDOWS 属性 --该属性仅支持 X 窗口系统的窗口管理器，其他系统均忽略
command(value=None)	--设置和获取 WM_COMMAND 属性 --该属性仅支持 X 窗口系统的窗口管理器，其他系统均忽略

deiconify()	--显示窗口 --默认情况下新创建的窗口都会显示在屏幕上，但是用 iconify()或 withdraw()方法可以在屏幕上移除窗口
focusmodel(model=None)	--设置和获取焦点模式
frame()	--返回一个字符串表示当前系统特征 --对于类 Unix 系统，返回值是 X 窗口标识符 --对于 Windows 系统，返回值是 HWND 强制转换为长整形的结果
geometry(geometry=None)	--设置和获取窗口的尺寸 --geometry 的参数格式为："%dx%d%+d%+d"%(width, height, xoffset, yoffset)
(wm_)grid(baseWidth=None, baseHeight=None, widthInc=None, heightInc=None)	--通知窗口管理器该窗口将以网格的形式重新调整尺寸 --baseWidth 和 baseHeight 指定 Tk_GeometryRequest 要求的网格单元数 --widthInc 和 heightInc 指定单元的高度和宽度
(wm_)group(window=None)	--将窗口添加到窗口群中 --window 参数指定控制窗口群的主窗口 --如果忽略该选项，将返回当前窗口群的主窗口
(wm_)iconbitmap(bitmap=None, default=None)	--设置和获取窗口的图标 --例如 root.iconbitmap(bitmap="python.ico") --default 参数可以用于指定由该窗口创建的子窗口的默认图标
(wm_)iconify()	--将窗口图标化（最小化） --需要重新显示窗口，用 deiconify()方法 --该方法会使得 state()返回"iconic"
(wm_)iconmask(bitmap=None)	--设置和获取位图编码

(wm_)iconname(newName=None)	--设置和获取当前窗口图标化（最小化）时的图标名字
(wm_)iconposition(x=None, y=None)	--设置和获取当前窗口图标化（最小化）时的图标位置
(wm_)iconwindow(pathName=None)	--设置和获取当前窗口图标化（最小化）时的组件窗口 --该方法会使得 state()返回“icon”
(wm_)maxsize(width=None, height=None)	--设置和获取该窗口的最大尺寸
(wm_)minsize(width=None, height=None)	--设置和获取该窗口的最小尺寸
(wm_)overrideredirect(boolean=None)	--如果参数为 True，该窗口忽略所有的小部件（也就是说该窗口将没有传统的标题栏、边框等部件）
(wm_)positionfrom(who=None)	--指定窗口位置由“谁决定 --如果 who 参数是“user”，窗口位置由用户决定 --如果 who 参数是“program”，窗口位置由系统决定
(wm_)protocol(name=None, func=None)	--将回调函数 func 与相应的规则 name 绑定 --name 参数可以是 “WM_DELETE_WINDOW”：窗口被关闭时候 --name 参数可以是“WM_SAVE_YOURSELF”：窗口被保存的时候 --name 参数可以是“WM_TAKE_FOCUS”：窗口获得焦点的时候
(wm_)resizable(width=None, height=None)	--指定是否可以改变该窗口的尺寸 --width 为 True 说明允许调整窗口的水平尺寸 --height 为 True 说明允许调整窗口的垂直尺寸

(wm_)sizefrom(who=None)	--指定窗口尺寸由“谁决定 --如果 who 参数是“user”，窗口尺寸由用户决定 --如果 who 参数是“program”，窗口尺寸由系统决定
(wm_)state(newstate=None)	--设置和获得当前窗口的状态 --newstate 的只可以是'normal', 'iconoc'(见 iconify), 'withdraw'(见 withdraw), 'icon'(见 iconwindow)和'zoomed'(放大，Windows 特有)
(wm_)title(string=None)	--设置窗口的标题
(wm_)transient(master=None)	--指定为 master 的临时窗口
(wm_)withdraw()	--将窗口从屏幕上移除（并没有销毁） --需要重新显示窗口，使用 deiconify()方法 --该方法会使得 state()返回"withdraw"
wm_aspect(minNumber=None, minDenom=None, maxNumber=None, manDenom=None)	-见上方 aspect()
wm_attributes(*args)	--见上方 attributes()
wm_client(name=None)	--见上方 client()
wm_colormapwindows(*wlist)	--见上方 colormapwindows()
wm_command(value=None)	--见上方 command()
wm_deiconify()	--见上方 deiconify()
wm_focusmodel(model=None)	--见上方 focusmodel()
wm_frame()	--见上方 frame()
wm_geometry(geometry=None)	--见上方 geometry()

attribute(*args)之 args 参数

选项	含义
alpha	（Windows，Mac）控制窗口的透明度 1.0 表示不透明，0.0 表示完全透明 该选项并不支持所有的系统，对于不支持的系统，Tkinter 绘制一个不透明（1.0）的窗口
disabled	（Windows）禁用整个窗口（这时候你只能从任务管理器中关闭它）
fullscreen	（Windows，Mac）如果设置为 True，则全屏显示窗口
modified	（Mac）如果设置为 True，该窗口被标记为改动过
titlepath	（Mac）设置窗口代理图标的路径
toolwindow	（Windows）如果设置为 True，该窗口采用工具窗口的样式
topmost	（Windows，Mac）如果设置为 True，该窗口将永远置顶

(十六)：Tkinter 的三大布局管理器 pack、grid 和 place

Tkinter 为我们提供了三个布局管理器：pack, grid 和 place

pack, grid 和 place 均用于管理同在一个父组件下的所有组件的布局，其中：

pack 是按添加顺序排列组件

grid 是按行列形式排列组件

place 则允许程序员指定组件的大小和位置

目录

- 何时使用 pack 管理器?
- 何时使用 grid 管理器?
- 如何使用 place 管理器:
- pack 的方法
  - pack(**options)
  - pack_configure(**options)
  - pack_forget()
  - pack_info()
  - pack_propagate(flag)
  - pack_slaves()
- grid 的方法:
  - grid(**options)
  - grid_bbox(column=None, row=None, col2=None, row2=None)
  - grid_columnconfigure(index, **options)
  - grid_configure(**options)
  - grid_forget()
  - grid_info()
  - grid_location(x, y)
  - grid_propagate(flag)
  - grid_remove()
  - grid_rowconfigure(index, **options)
  - grid_size()
  - grid_slaves(row=None, column=None)
- place 的方法

place(**options)
place_configure(**options)
place_forget()
place_info()
place_slaves()
slaves()

何时使用 pack 管理器?

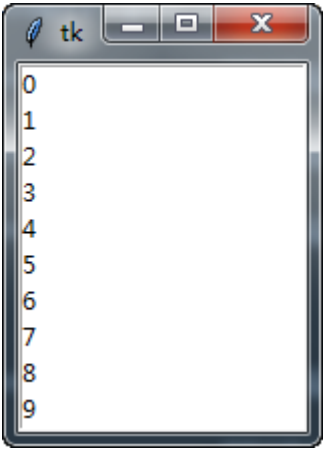
对比 grid 管理器，pack 更适用于少量组件的排列，但它在使用上更加简单（就像前面的所有例子中，展示一个组件我们一般都直接使用 .pack() 方法）。如果你需要创建相对复杂的布局结构，那么建议是使用多个框架（Frame）结构构成，或者使用 grid 管理器实现。

注意：不要在同一个父组件中混合使用 pack() 和 grid()，因为 Tkinter 无法确定首先使用哪个布局管理器。

我们常常会遇到的一种情况是将一个组件放到一个容器组件中，并填充整个父组件。下面举个生成一个 Listbox 组件并将它填充到 root 窗口中：

```
1. from tkinter import *
2.
3. root = Tk()
4.
5. listbox = Listbox(root)
6. listbox.pack(fill=BOTH, expand=True)
7. #fill 选项是填充整个父组件，expand 选项是拉伸时依旧填满
8.
9. for i in range(10):
10.     listbox.insert(END, str(i))
11.
12. mainloop()
```

其中，fill 选项是告诉窗口管理器该组件将填充整个分配给它的空间，BOTH 表示同时横向和纵向扩展，X 表示横向，Y 表示纵向。expand 选项是告诉窗口管理器将父组件的额外空间也充满。



默认下，pack 是将添加的组件一次纵向排列

```
1. from tkinter import *
2.
3. root = Tk()
4.
5. Label(root, text='red', bg='red', fg='white').pack(fill=X) #fill 选项是设置横向填充
6. Label(root, text='red', bg='green', fg='black').pack(fill=X)
7. Label(root, text='red', bg='blue', fg='white').pack(fill=X)
8. #整体是纵向填充
9.
10. #Label(root, text='red', bg='red', fg='white').pack(side=LEFT) 如果想将组件整体横向排列，可以设置 side 选项
11.
12. mainloop()
```

何时使用 grid 管理器?

grid 管理器可以说是 Tkinter 这三个布局管理器中最灵活多变的。如果你只希望学习使用一个布局管理器，那么 grid 绝对是首选。当你在设计对话框的时候，使用 grid 尤其便捷。使用一个 grid 就可以简单的实现你用很多个框架和 pack 搭建起来的效果。

使用 grid 排列组件，只要告诉它你想要组件放置的位置（行 / 列，row 选项指定行，column 选项指定列）。此外你并不用提前指出网格（grid 分布给组件的位置称为网格）的尺寸，因为管理器会自动计算。

设置一个简单的登录界面：

```
1. from tkinter import *
2.
3. root = Tk()
4.
5. Label(root, text='用户名
   ').grid(row=0, column=0, sticky=W)  #sticky 选项设置组件
   朝向
6. Label(root, text='密码
   ').grid(row=1, column=0, sticky=W)
7.
8. Entry(root).grid(row=0, column=1)
9. Entry(root, show='*').grid(row=1, column=1)
10.
11. mainloop()
```

用几个网格来放置一个组件，可使用 rowspan 和 columnspan 实现跨行跨列的功能：

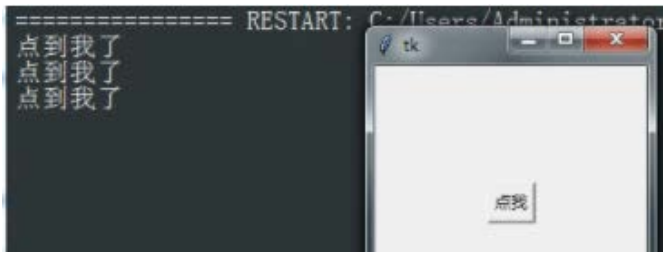
```
1. from tkinter import *
2. 
3. root = Tk()
4. 
5. Label(root, text='用户名
   ').grid(row=0, column=0, sticky=W)
6. Label(root, text='密码
   ').grid(row=1, column=0, sticky=W)
7. 
8. photo = PhotoImage(file=r"D:\u.gif")  #只能是.gif 格式
9. Label(root, image=photo).grid(row=0, column=2, rowspan=
   2, padx=5, pady=5)  #rowspan 选项设置了跨行，现在 photo 组件
   可以显示 0,1 两行了。padx，pady 设置了边距，使图片文字不挨在一
   起
10. 
11. Entry(root).grid(row=0, column=1)
12. Entry(root, show='*').grid(row=1, column=1)
13. 
14. def test():
15.     print('提交成功')
16. 
17. Button(root, text='提交
   ', width=10, command=test).grid(row=2, columnspan=3, pa
   dy=5)  #column 默认是 0
18. 
19. mainloop()
```

如何使用 place 管理器：

通常情况下不建议使用 place 布局管理器，因为对比起 pack 和 grid，place 要做更多的工作。不过存在即合理，place 在一些特殊的情况下可以发挥妙用，甚至是 pack 和 grid 无法代替。

将子组件显示在父组件的正中间

```
1. from tkinter import *
2.
3. root = Tk()
4.
5. def callback():
6.     print('点到我了')
7.
8. Button(root, text='点我
   ', command=callback).place(relx=0.5, rely=0.5, anchor=C
   ENTER)  #relx 和 rely 是相对父组件的位置。0 是最左边，0.5 是正
   中间，1 是最右边
9.
10. mainloop()
```



在某种情况下，你希望一个组件能够覆盖另一个组件。下面使用一个按钮覆盖一个 Label 图片的例子

```
1. from tkinter import *
2.
3. root = Tk()
4.
5. photo = PhotoImage(file=r"D:\u.gif")
6. Label(root, image=photo).pack()
7.
8. def callback():
9.     print('点到我了')
10.
11. Button(root, text='点我', command=callback).place(relx=0.5, rely=0.5, anchor=CENTER)  #relx 和 rely 是相对父组件的位置。0 是最左边，0.5 是正中间，1 是最右边
12.
13. mainloop()
```

relx 和 rely 选项指定的是相对于父组件的位置，范围是 00~1.0
relheight 和 relwidth 选项则是指定相对于父组件的尺寸

```
1. from tkinter import *
2.
3. root = Tk()
4.
5. Label(root, bg='red').place(relx=0.5, rely=0.5, relheig
   ht=0.75, relwidth=0.75, anchor=CENTER)  #relheight 和
   relwidth 是设置调用 place 的子组件相对于父组件的高度和宽度
6. Label(root, bg='yellow').place(relx=0.5, rely=0.5, relh
   eight=0.5, relwidth=0.75, anchor=CENTER)
7. Label(root, bg='green').place(relx=0.5, rely=0.5, relhe
   ight=0.25, relwidth=0.75, anchor=CENTER)
8.
9. mainloop()
```

还有 x 和 y 选项用于设置偏移(像素)，如果同时设置 relx(rely) 和 x(y)，那么 place 将优先计算 relx 和 rely，然后再实现 x 和 y 指定的偏移值。

pack 的方法

注：下边方法适用于所有组件

pack(**options)

--下方详细列举了各个选项的具体含义和用法

选项	含义
anchor	--控制组件在 pack 分配的空间中的位置 --N, NE, E, SE, S, SW, W, NW 或 CENTER 来定位（EWSN 表示东南西北） --默认值是 CENTER
expand	--指定是否填充父组件的额外空间 --默认值是 False
fill	--指定填充 pack 分配的空间 --默认值是 NONE，表示保持子组件的原始尺寸 --还可以使用的值有：X（水平填充），Y（垂直填充）和 BOTH（水平和垂直填充）
in_	--将该组件放到该选项指定的组件中 --指定的组件必须是该组件的父组件
ipadx	--指定水平方向上的内边距
ipady	--指定垂直方向上的内边距
padx	--指定水平方向上的外边距
pady	--指定垂直方向上的外边距
side	--指定组件的放置位置 --默认值是 TOP --还可以设置的值有：LEFT，BOTTOM，RIGHT

pack_configure(**options)

--跟 pack() 一样

pack_forget()

--将组件从屏幕中“删除”

--并没有销毁该组件，只是看不到了

--可以通过 pack 或其他布局管理器显示已“删除”的组件

pack_info()

--以字典的形式返回当前的 pack 的选项

pack_propagate(flag)

--如果开启，父组件会自动调节尺寸以容纳所有子组件

--默认值是开启（flag=True）

--该方法仅适用于父组件

pack_slaves()

--以列表形式返回该组件的所有子组件

--该方法仅适用于父组件

grid 的方法：

注：下边方法适用于所有组件

grid(options)**

--下面列举了各个选项的具体含义和用法

选项	含义
column	--指定组件插入的列（0 表示第一列） --默认值是 0
columnspan	--指定用多少列（跨列）显示该组件
in_	--将该组件放到该选项指定的组件中 --指定的组件必须是该组件的父组件
ipadx	--指定水平方向上的内边距
ipady	--指定垂直方向上的内边距
padx	--指定水平方向上的外边距
pady	--指定垂直方向上的外边距
row	--指定组件插入的行（0 表示第一行）
rowspan	--指定用多少行（跨行）显示该组件
sticky	--控制组件在 grid 分配的空间中的位置 --可以使用 N，E，S，W 以及他们的组合来定位 --使用加号（+）表示拉长填充，例如 N+S 表示将该组件垂直拉长填充网格，N+S+W+E 表示填充整个网格 --不指定该值则居中显示

grid_bbox(column=None, row=None, col2=None, row2=None)

--返回一个四元组描述该组件所在的限定矩形

--如果指定 column 和 row 参数，则返回该位置（column, row）的组件的限定矩形的描述

grid_columnconfigure(index, **options)

--设置列的属性

--注意：设置的是该组件所拥有的 grid 序列

--可以设置的选项及含义如下：

选项	含义
minsize	--指定该列的最小宽度
pad	--指定该列中最大网格的水平边距
weight	--指定列与列之间的相对距离 --默认值是 0 --说明：初创建窗口的时候，grid 会自动根据组件的尺寸分配窗口的尺寸，当你拉伸窗口的尺寸就会有空白显示出来。这个选项正是指定列与列之间是否填充空白，默认是不填充的。另外，该选项的值是指定填充空白的倍数，例如 weight=2 的列会比 weight=1 的列填充多一倍的空白，所以需要平均填充的话，只需要所有的列都设置为 weight=1 即可。

grid_configure(options)**

--跟 grid() 一样

grid_forget()

--将组件从屏幕中“删除”

--并没有销毁该组件，只是看不到了

--可以通过 grid 或其他布局管理器显示已“删除”的组件，但该组件所在网格的选项设置不会恢复

grid_info()

--以字典的形式返回当前 grid 的选项

grid_location(x, y)

--返回位于（或接近）给定坐标（x, y）的网格位置

--返回值是一个 2 元组表示网格对应的（列， 行）

grid_propagate(flag)

--如果开启，父组件会自动调节尺寸以容纳所有子组件

--默认值是开启（flag=True）

--该方法仅适用于父组件

grid_remove()

--跟 grid_forget() 一样，但恢复的时候会记住该组件所在网格的选项设置

grid_rowconfigure(index, **options)

--设置行的属性

--注意：设置的是该组件所拥有的 grid 的行

--可以设置的选项及含义如下：

选项	含义
minsize	-指定该行的最小高度
pad	--指定该列中最大网格的垂直边距
weight	--指定行于行之间的相对距离 --默认值是 0

grid_size()

--返回该组件所拥有的 grid 的尺寸

--返回值是一个 2 元组，表示（列，行）分别的网格数

grid_slaves(row=None, column=None)

--以列表的形式返回该组件的所有子组件

--该方法仅适用于父组件

place 的方法

注：下边的方法适用于所有的组件

place(options)**

--下方列举了各个选项的含义和用法

选项	含义
anchor	--控制组件在 place 分配的空间中的位置 --N, NE, E, SE, S, SW, W, NW 或 CENTER 来定位（EWSN 表示东南西北） --默认值是 NW
bordermode	-指定边框模式（INSIDE 或 OUTSIDE） --默认值是 INSIDE
height	--指定该组件的高度（像素）
in_	--将该组件放到该选项指定的组件中 --指定的组件必须是该组件的父组件
relheight	--指定该组件相对于父组件的高度 --取值范围是 0.0~1.0
relwidth	--指定该组件相对于父组件的宽度 --取值范围是 0.0~1.0
relx	--指定该组件相对于父组件的水平位置 --取值范围是 0.0~1.0
rely	--指定该组件相对于父组件的垂直位置 --取值范围是 0.0~1.0
width	-指定该组件的宽度（像素）
x	-指定该组件的水平偏移位置（像素） --如果同时指定了 relx 选项，优先实现 relx 选项
y	--指定该组件的垂直偏移位置（像素） --如果同时指定了 rely 选项，优先实现 rely 选项

place_configure(options)**

--跟 place()一样

place_forget()

--将组件从屏幕中“删除”

--并没有销毁该组件，只是看不到了

--可以通过 place 或其他布局管理显示已“删除”的组件

place_info()

--以字典的形式返回当前 place 的选项

place_slaves()

--以列表的形式返回该组件的所有子组件

--该方法仅适用于父组件

slaves()

--跟 place_slaves() 一样

(十七)：Tkinter 的标准对话框模块 messagebox 、filedialog 和 colorchooser

Tkinter 为我们提供了三种标准对话框模块，他们分别是：messagebox、filedialog 和 colorchooser

注：这三个模块原来是独立的，分别是 tkMessageBox、tkFileDialog 和 tkColorChooser，需要导入才能使用。在 Python3 之后，这些模块全部被归纳到 Tkinter 模块的麾下。

目录

Messagebox（消息对话框）

函数参数

函数返回值

filedialog（文件对话框）

函数参数

函数返回值

colorchooser（颜色对话框）

函数参数

函数返回值

FAQ

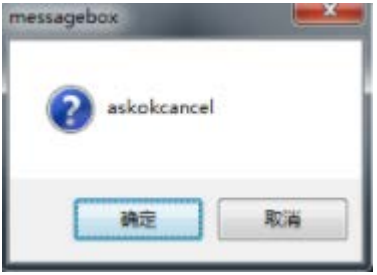
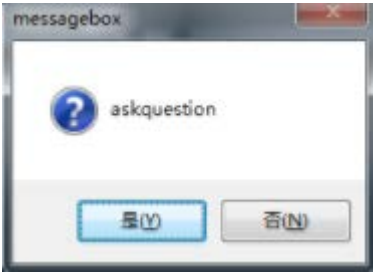
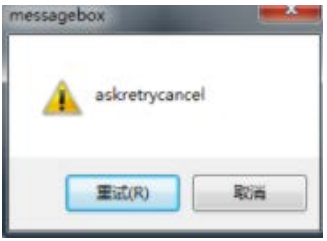
Messagebox（消息对话框）

下文为你列出了使用 messagebox 可以创建的所有标准对话框样式：

```
1. from tkinter import messagebox
2. #import tkinter.messagebox
3. from tkinter import *
4.
5. print(messagebox.askokcancel('messagebox', 'Demo one'))
```

```
#有返回值，点‘确定’返回 True，点‘取消’返回 False。后序可以有对返回值的操作进行下一步

mainloop()
```

使用函数和对话框样式

askokcancel(item, message, options)	
askquestion(title, message, options)	
askretrycancel(title, message, options)	
askyesno(title, message, options)	

showerror(title, message, options)	
showinfo(title, message, options)	
showwarning(title, message, options)	

函数参数

所有的这些函数都有相同的参数：

title 参数设置标题栏的文本
message 参数设置对话框的主要内容，可以使用'\n'来进行换行
options 参数可以设置的选项和含义如下表

options 参数选项	含义
default	--设置默认的按钮（也就是按下回车响应的那个按钮）

	--默认是第一个按钮（像‘确定’，‘是’或‘重试’） --可以设置的值根据对话框的不同可以进行选择：CANCEL、IGNORE、OK、NO、RETRY 或 YES
icon	--指定对话框显示的图标 --可以指定的值有：ERROR、INFO、QUESTION 或 WARNING --注意：不能自己指定自己的图标（不可自定义图标）
parent	--如果不指定该选项，那么对话框默认显示在根窗口上 --如果想要讲对话框显示在子窗口 w 上,那么可以设置 parent=w

函数返回值

askokcancel()、askretrycancel() 和 askyesno() 返回布尔类型的值：

--返回 True 表示用户点击了 ‘确定’ 或 ‘是’ 按钮

--返回 False 表示用户点击了 ‘取消’ 或 ‘否’ 按钮

askquestion() 返回 ‘yes’ 或 ‘no’ 字符串表示用户点了‘是’ 或 ‘否’ 按钮

showerror()、showinfo() 和 showwarning() 返回 ‘ok’ 表示用户按下‘是’按钮

filedialog（文件对话框）

当你的应用程序需要使用打开文件或保存文件的功能时，文件对话框显得尤为重要。emmm，印象里 Java 好像也有个组件还是函数可以几行实现。

```
#from tkinter import filedialog
import tkinter.filedialog
from tkinter import *

root = Tk()

def callback():
    fileName = filedialog.askopenfilename()
    print(fileName)

Button(root, text='打开文件', command=callback).pack()

mainloop()
```

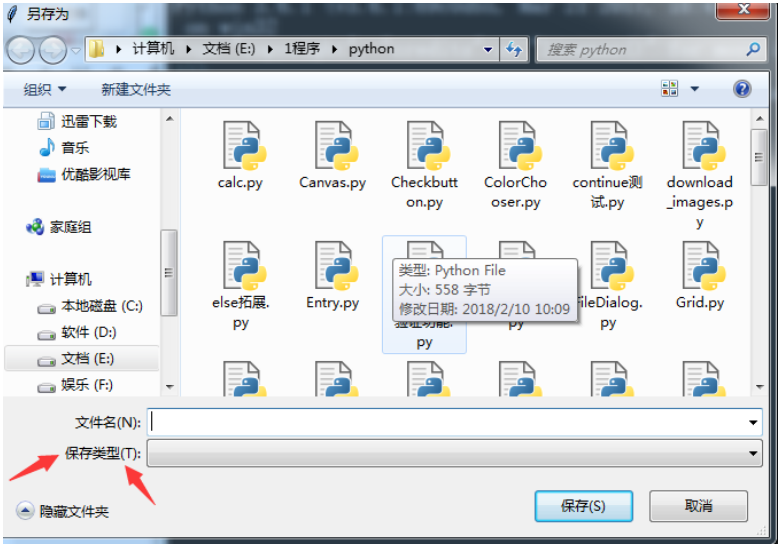
filedialog 模块提供了两个函数：askopenfilename(**options) 和 asksaveasfilename(**options)，分别用于打开文件和保存文件。

函数参数

两个函数可供设置的选项是一样的，下面列举了可用选项及含义：

可用选项	含义
defaultextension	--指定文件的后缀 --例如：defaultextension='.jpg'，那么当用户输入一个文件名'Python'的时候，文件名会自动添加后缀为'Python.jpg' --注意：如果用户输入文件名包含后缀，那么该选项不生效
filetypes	--指定筛选文件类型的下拉菜单选项 --该选项的值是由二元组构成的列表 --每个二元组是由 (类型名, 后缀) 构成, 例如: filetypes=[('PNG', '.png'), ('JPG', '.jpg'), ('GIF', '.gif')]
initialdir	--指定打开保存文件的默认路径 --默认路径是当前文件夹
parent	--如果不指定该选项，那么对话框默认显示在根窗口上 --如果想要将对话框显示在子窗口 w 上,那么可以设置 parent=w
title	--指定文件对话框的标题栏文本

filetypes 图解:



函数返回值

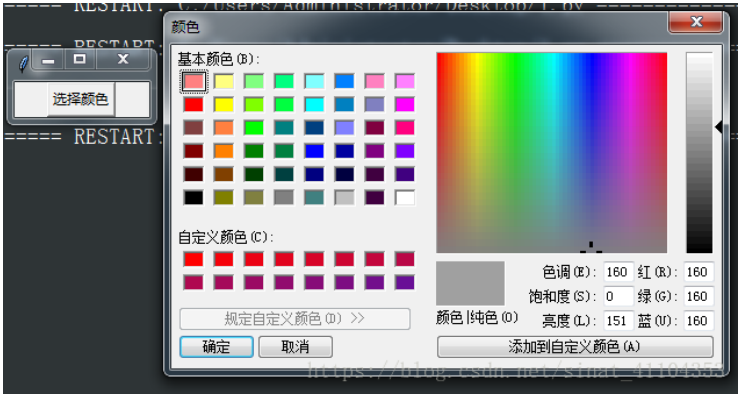
1.如果用户选择了一个文件，那么返回值是该文件的完整路径

2.如果用户点击了取消按钮，那么返回值是空字符串

colorchooser（颜色对话框）

颜色对话框提供了一个友善的界面让用户选择需要的颜色。

```
1. from tkinter import colorchooser
2. from tkinter import *
3.
4. root =Tk()
5.
6. def callback():
7.     fileName = colorchooser.askcolor()
8.     print(fileName)
9.
10. Button(root, text='选择颜色', command=callback).pack()
11.
12. mainloop()
```



colorchooser 模块提供了一个函数 askcolor(color, option=value, ...)

函数参数

参数	含义
color	--要显示的初始的颜色(下图四个红箭头指的地方) --默认颜色是浅灰色（light gray）
parent	--如果不指定该选项，那么对话框默认显示在根窗口上 --如果想要将对话框显示在子窗口 w 上，那么可以设置 parent=w
title	--指定颜色选择器标题栏的文本 --默认标题是“颜色”

“默认颜色”图解：

函数返回值

1. 如果用户点击的 ‘确定’ 按钮，返回值是一个二元组（triple, color），其中的 triple 是一个三元组（R, G, B)--其中 R, G, B 的范围是 [0, 255]（就是该颜色的 RGB 颜色），第二个参数选中颜色的 16 进制的值
2. 如果用户点击的 ‘取消’ 按钮，返回值是（None, None）

Q&A

可能有人会比较好奇为什么要这样写：

```
1. from tkinter import colorchooser
2. from tkinter import *
```

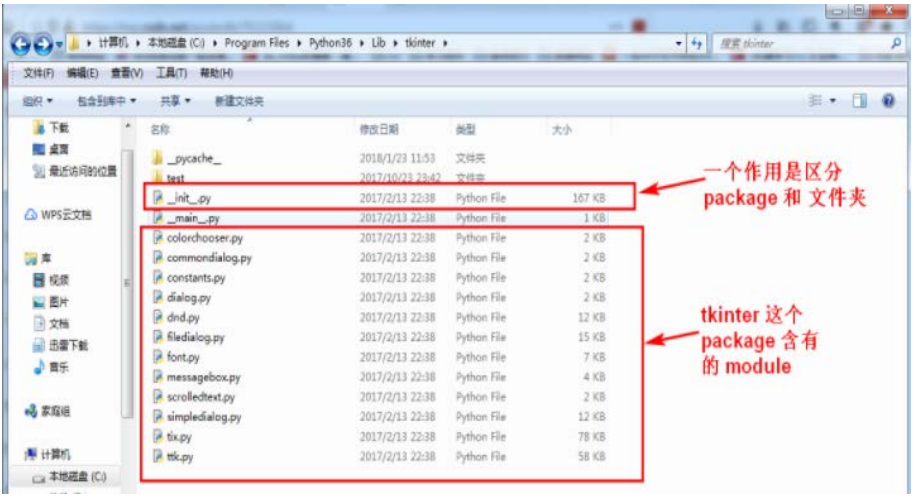
其中的 from tkinter import * 难道没有包含 from tkinter import colorchooser 吗？

经过尝试我们会发现，还必须加上 from tkinter import colorchooser 这段代码，否则会报错。

首先我们得对 Python 的 module 和 package 这两个概念有所了解，看 sof 上的这个问题：difference between a Python module and a Python package

简单归纳如下

> 任何一个 Python 文件都是一个都是一个 module，module 名称是没有 .py 扩展名的那部分。比如说有一个 a.py 的 Python 文件，那么这个文件就是一个 module，module 的名称是 a。我们可以通过 import a 来导入这个 module。
>
> package 是很多 module 的集合。可以将 package 看成一个文件夹。将 package 和一个含有很多 .py 文件的文件夹区分开来的标准是 package 还有一个额外的 __init__.py。



可以参考 sof 上的这个提问： whats the diference between from tkinter import * and from tkinter import 'module'

下面是我对第一个链接最高赞的一个简单的概述：

当我们 import package 的时候，只是 import 了 package 的 __init__.py 中的 variables/functions/classes，而没有 import package 包含的 module。

所以如果我们想要使用 module 中的变量、函数、类，就得具体指明是哪个 module。